UPIITA IPN

Dispositivos Lógicos Programables

Proyecto DLP: Laberinto

13/06/2023

Equipo 1

Integrantes:

- Espinoza Flores Giovanni
- Melo Romo Anette Cheyenne
- Rendón Chavez Miguel Abraham

Grupo: 2MV23

RESUMEN:

En este proyecto, el objetivo es implementar los conocimientos que se han adquirido a lo largo de la trayectoria de la Unidad de Aprendizaje de Dispositivos Lógicos Programables, desde el uso de electrónica para el armado de circuitos de implementación utilizando distintos componentes y periféricos que se encuentran disponibles en la tarjeta de desarrollo de dispositivos programables (FPGA) mediante el lenguaje de programación VHDL. A lo largo de la realización de este proyecto se aprendió a integrar componentes de salida para la visualización de información, pantalla LCD en este caso, con servomotores (sg90) controlados mediante un módulo PmodACL.

ABSTRACT

In this project, the objective is to implement the knowledge that has been acquired throughout the trajectory of the Programmable Logic Devices Learning Unit, from the use of electronics to the assembly of implementation circuits using different components and peripherals that are found available on the programmable device development board (FPGA) using the VHDL programming language. Throughout the realization of this project, it was learned to integrate output components for the visualization of information, LCD screen in this case, with servomotors (sg90) controlled by a PmodACL module

ZUSAMMENFASSUNG

In diesem Projekt besteht das Ziel darin, das Wissen umzusetzen, das im Laufe der Lerneinheit, Programmierbare Logikgeräte“ erworben wurde, von der Verwendung der Elektronik bis zum Zusammenbau von Implementierungsschaltungen unter Verwendung verschiedener Komponenten und Peripheriegeräte, die auf dem programmierbaren Gerät verfügbar sind Entwicklungsboard (FPGA) mit der Programmiersprache VHDL. Im Laufe der Umsetzung dieses Projekts wurde gelernt, Ausgabekomponenten für die Visualisierung von Informationen, in diesem Fall LCD-Bildschirme, mit Servomotoren (sg90) zu integrieren, die von einem PmodACL-Modul gesteuert werden.

INDICE:

INTRODUCCIÓN: 4
CIRCUITO ESQUEMATICO 6
DIAGRAMA DE FLUJO O BLOQUE 7
SIMULACIÓN 10
CÓDIGOS 11
FOTOS 61
CONCLUSIONES 66
BIBLIOGRAFIA 67

INTRODUCCIÓN:

Un módulo Pmod ACL (Acelerómetro) es un dispositivo de interfaz diseñado para medir la aceleración en tres ejes (X, Y y Z). Estos módulos son utilizados en aplicaciones que requieren detectar cambios en la velocidad o dirección del movimiento. Pmod ACL se conecta a una plataforma de desarrollo o microcontrolador a través de un conector Pmod estándar.

Nombre: Pmod ACL (Acelerómetro) Fabricante: Digilent Número de modelo: 410-247

Especificaciones técnicas:

- Acelerómetro: ADXL345 de Analog Devices
- Rango de medición: ±2 g, ±4 g, ±8 g, ±16 g
- Resolución: 10 bits
- Comunicación: Protocolo I2C
- Tasa de datos: Hasta 3.2 kHz
- Alimentación: 3.3V

Características adicionales:

- Salida de datos en formato digital
- Detección de movimiento y vibración
- Detección de orientación
- Baja corriente de operación

El módulo Pmod ACL proporciona una interfaz fácil de usar para integrar la funcionalidad de acelerómetro en proyectos de desarrollo electrónico. Al utilizar el protocolo I2C, es posible comunicarse con el módulo a través de pocos pines y obtener datos precisos sobre la aceleración en tiempo real. Esto hace que sea ideal para aplicaciones como monitoreo de movimiento, detección de caídas, juegos interactivos y más.

El servomotor SG90 es un pequeño motor utilizado en aplicaciones de control de movimiento, como robots, vehículos teledirigidos, sistemas de automatización y proyectos electrónicos en general. Está diseñado para proporcionar un control preciso del ángulo de giro y es ampliamente utilizado debido a su

tamaño compacto, bajo costo y facilidad de uso.

- Modelo: SG90
- Voltaje de funcionamiento: 4.8V - 6V
- Corriente en reposo: 10mA
- Velocidad de operación: 0.12 segundos/60 grados (sin carga)
- Rango de ángulo de giro: 180 grados
- Torque: 1.8 kg/cm (4.8V), 2.2 kg/cm (6V)
- Tipo de motor: Motor de corriente continua (DC)
- Tipo de control: Control de posición por señales PWM (Modulación por Ancho de Pulso)
- Dimensiones: 23 mm x 12.2 mm x 29 mm
- Peso: 9 gramos

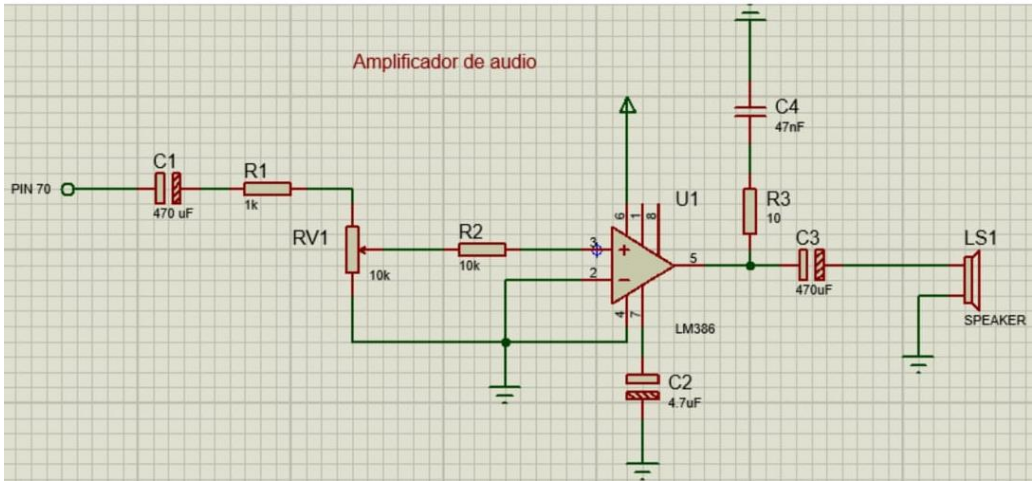
Características adicionales:

- Engranajes de plástico para reducir el ruido y aumentar la durabilidad
- Conexión de tres cables: señal de control (generalmente color amarillo), alimentación (rojo) y tierra (negro)
- Ángulo de giro preciso y repetible
- Fácilmente interfaz con microcontroladores y placas de desarrollo

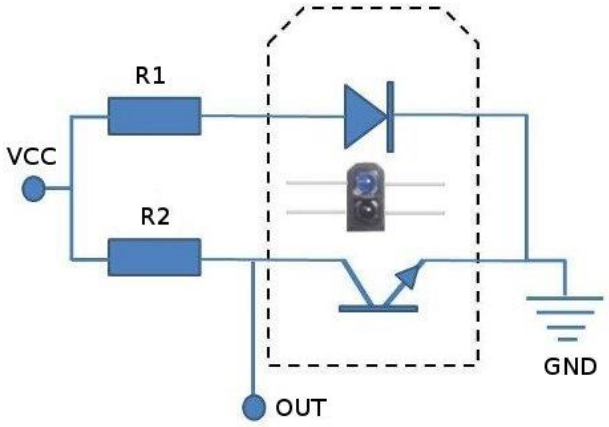
El servomotor SG90 es conocido por su tamaño compacto, bajo consumo de energía y capacidad para girar con precisión en un rango de 180 grados. Es ampliamente utilizado en proyectos de robótica y electrónica donde se requiere un control preciso del movimiento. Al enviar señales PWM adecuadas a través del cable de control, es posible controlar la posición del servomotor y lograr movimientos suaves y controlados.

CIRCUITO ESQUEMATICO

Circuito de potencia para cada una de las bocinas (se usan 2, ya que usamos dos grabadores de sonido y 2 bocinas)



Para cuando se pierde se usa un sensor justo en donde regresa la pelota, este sensor se encarga de detectar el paso y cuando lo detecta se envia la señal a la FPGA para así cambiar a la música y mensaje de que se perdió el juego



Fuera de estos circuitos no se usó nada más, se conectó directamente el acelerómetro, así como la LCD a la Amiba

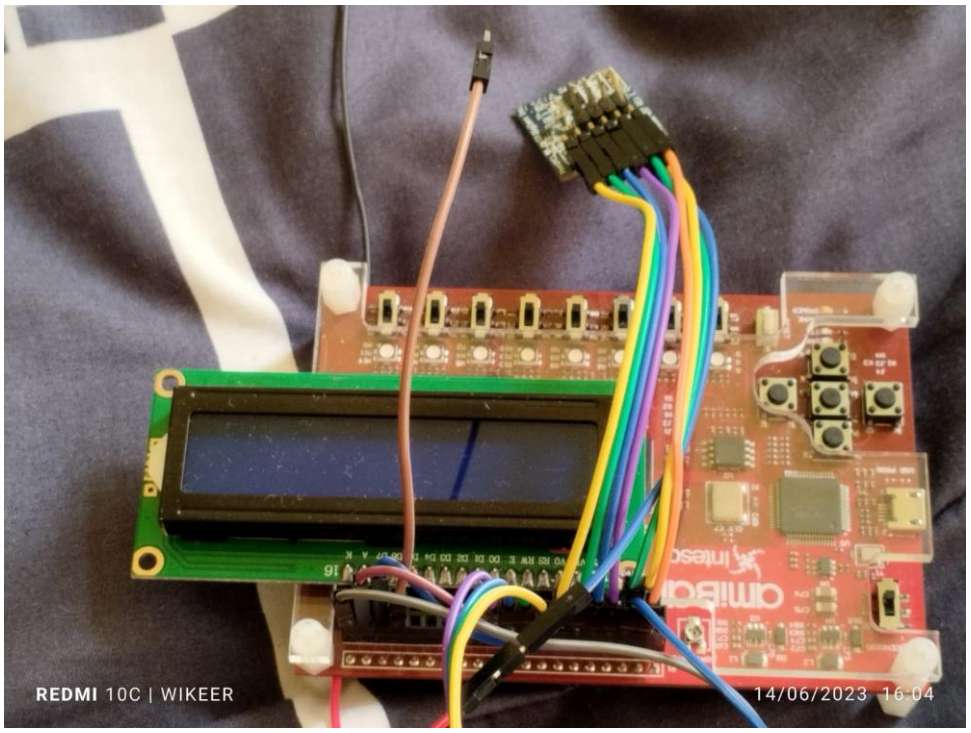
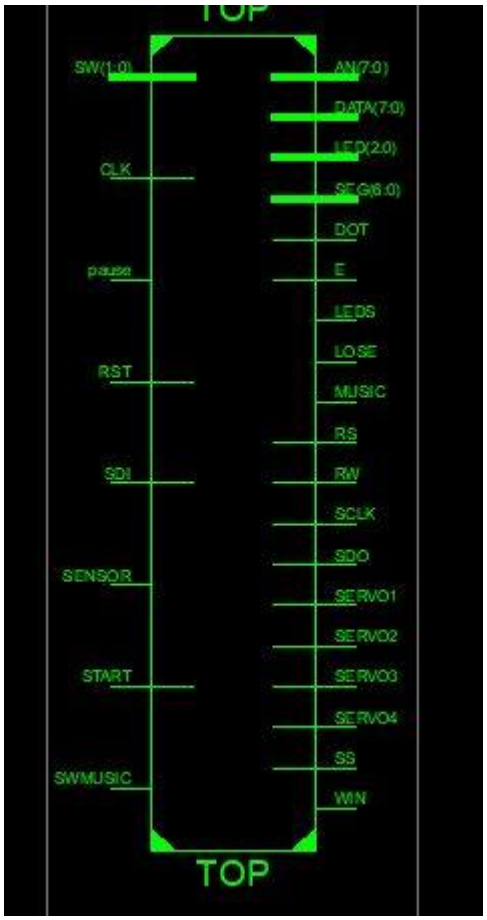
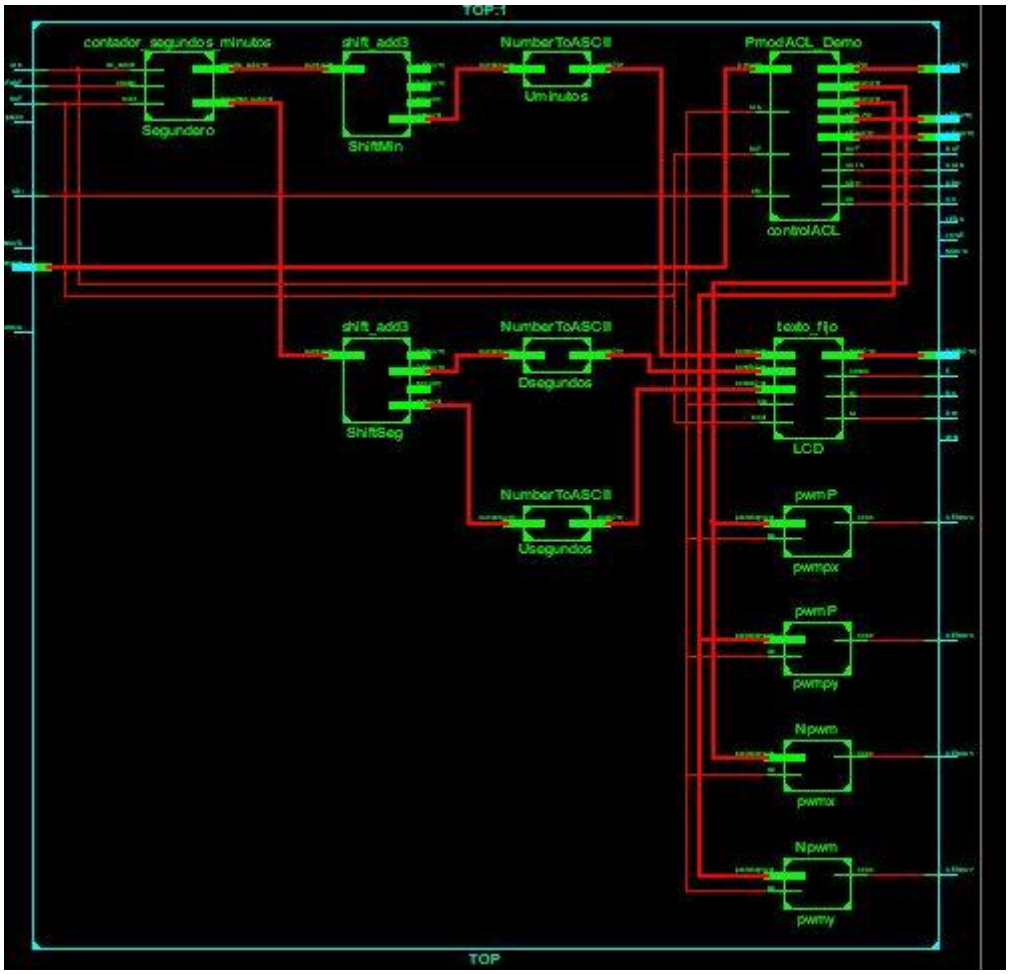
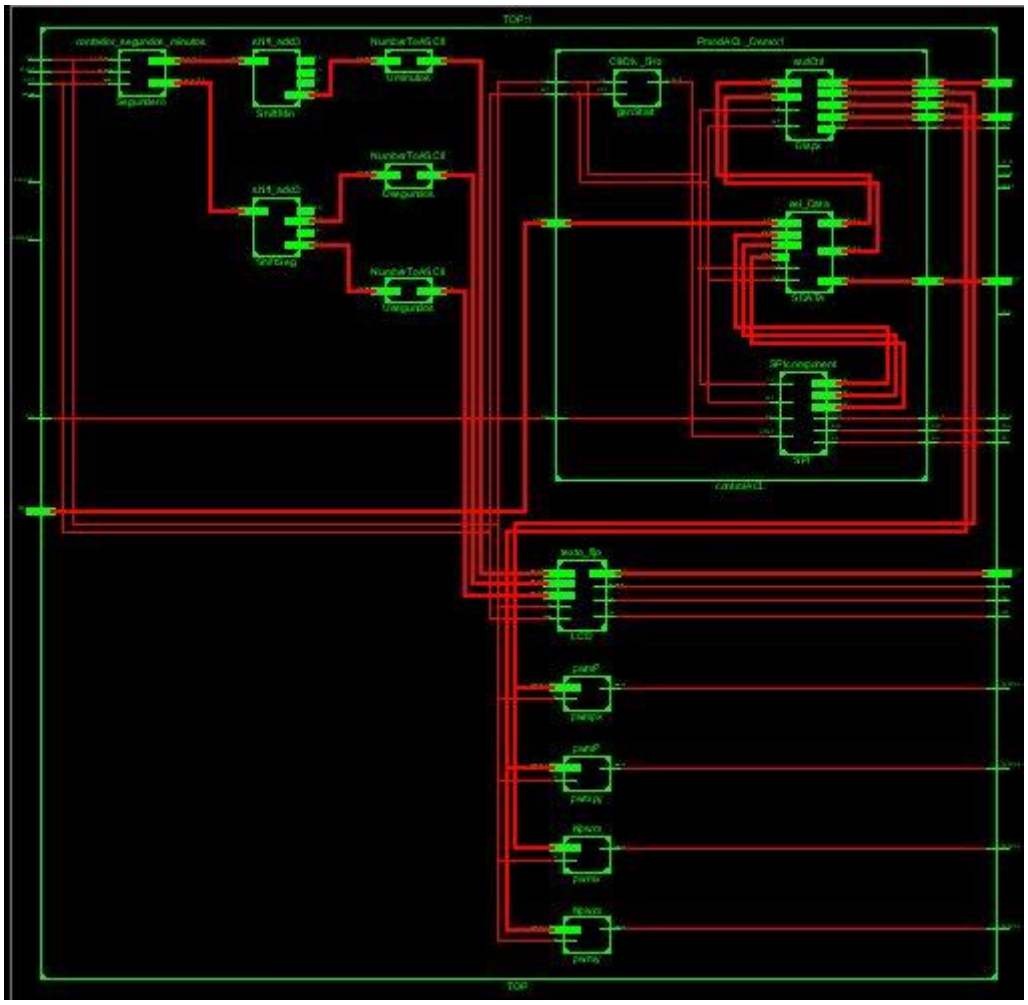
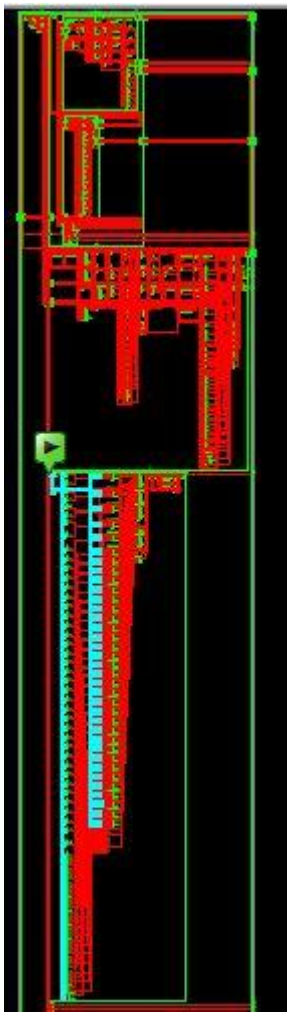


DIAGRAMA DE FLUJO O BLOQUE

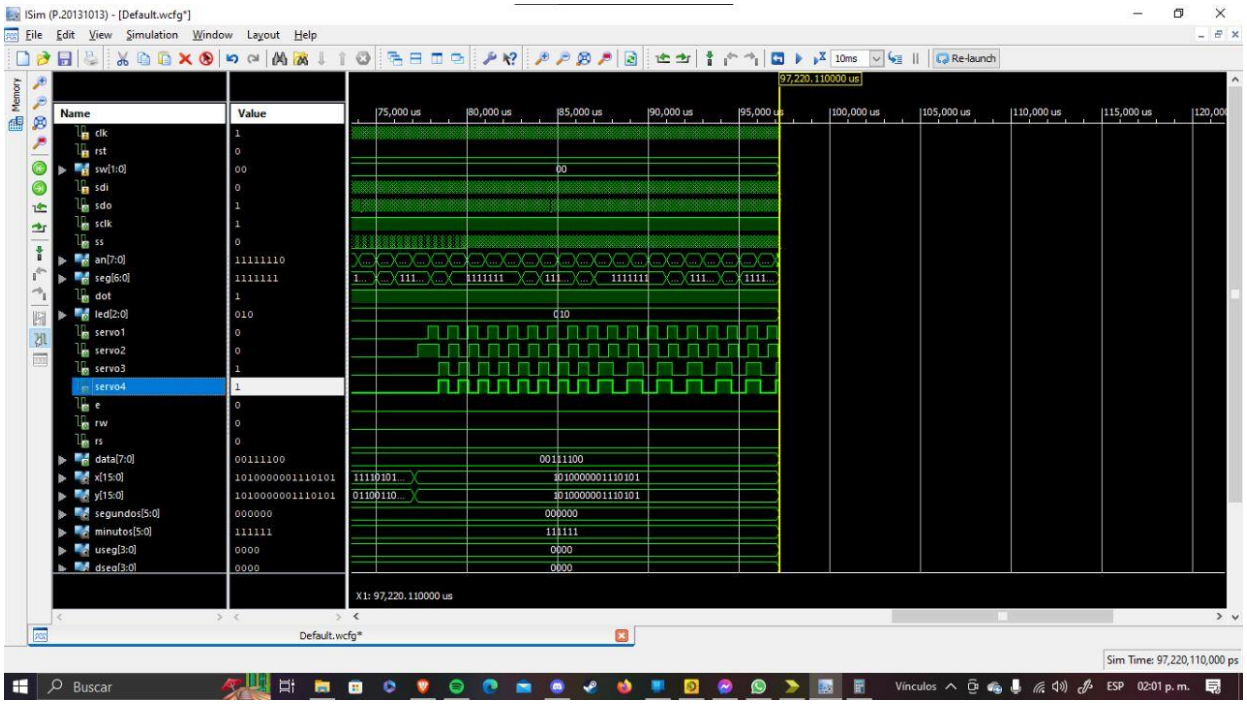



TOP-1
contador_segundos_minutos
Segundoro
shift_add3
ShiftMin
NumberToASCII
Uminutos
PmodACL_Demo
controlACL
shift_add3
ShiftSeg
NumberToASCII
Dsegundos
NumberToASCII
Usegundos
texto_fijo
LCD
pwmP
pwmpx
pwmP
pwmpy
Npwm
pwmx
Npwm
pwmy
TOP



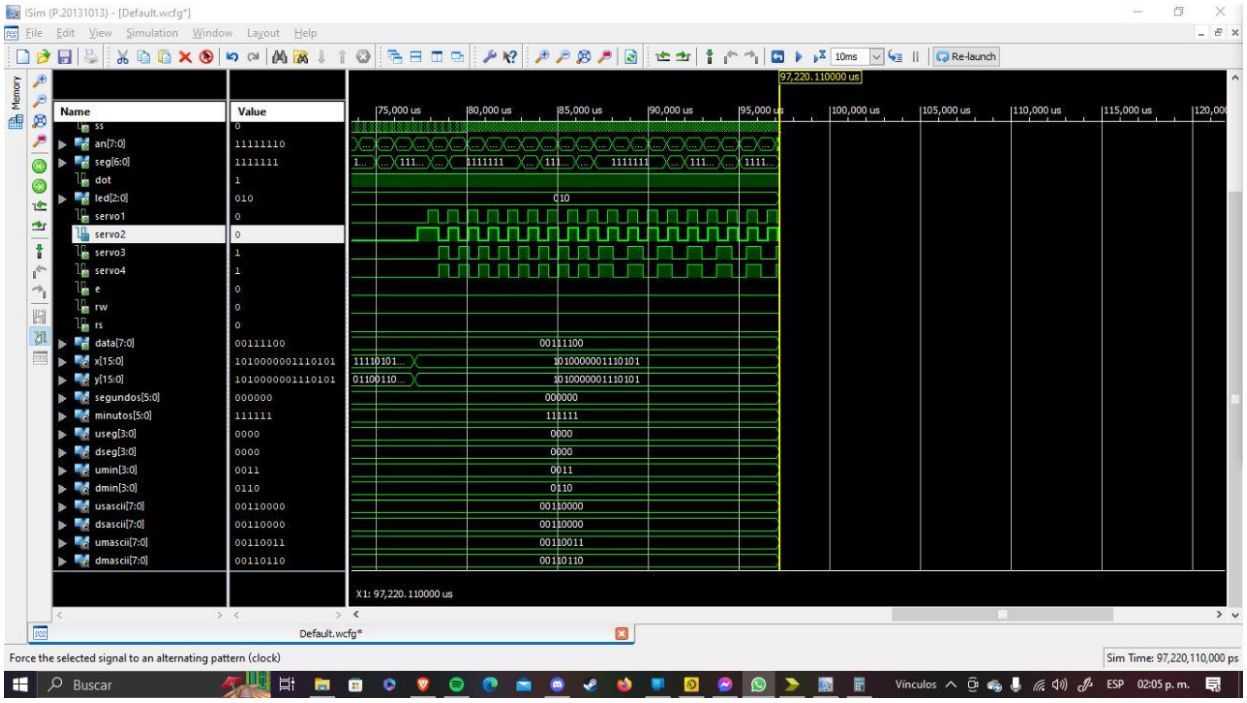


SIMULACIÓN


ISim (P.20131013) - [Default.wcfg*]
File Edit View Simulation Window Layout Help
Name
Value
clk 1
rst 0
sw[1:0] 00
sdi 0
sdo 1
sclk 1
ss 0
an[7:0] 11111110
seg[6:0] 1111111
dot 1
led[2:0] 010
servo1 0
servo2 0
servo3 1
servo4 1
e 0
rw 0
rs 0
data[7:0] 00111100
x[15:0] 1010000001110101
y[15:0] 1010000001110101
segundos[5:0] 000000
minutos[5:0] 111111
useg[3:0] 0000
dseg[3:0] 0000
X1: 97,220,110000 us
Default.wcfg*
Sim Time: 97,220,110,000 ps

Se pude observar como la entrada correspondiente a 1010000001110101 que seria 75 para lo que tendría un estado alto de 0.0111s

Mientas que los opuestos es de 0.00163336s para 75 grados



CÓDIGOS

TLD

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use IEEE.std_logic_arith.all;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
entity TOP is
   Port ( CLK : in  STD_LOGIC;--
          RST : in  STD_LOGIC;--
          SW : in  STD_LOGIC_VECTOR (1 downto 0);--
          SDI : in  STD_LOGIC;--
          SDO : out  STD_LOGIC;--
          SCLK : out  STD_LOGIC;--
          SS : out  STD_LOGIC;--
          AN : out  STD_LOGIC_VECTOR (7 downto 0);--
          SEG : out  STD_LOGIC_VECTOR (6 downto 0);--
          DOT : out  STD_LOGIC;--punto--
          LED : out  STD_LOGIC_VECTOR (2 downto 0);--
                                SERVO1 : out STD_LOGIC;--
                                SERVO2 : out STD_LOGIC;--
                                SERVO3 : out STD_LOGIC;--
                                SERVO4 : out STD_LOGIC;--
                                E : out STD_LOGIC;--
                                RW : OUT STD_LOGIC;--
```

```
                        RS : out STD_LOGIC;--
                        DATA : out std_logic_vector(7 downto 0);--
                        LOSE : out STD_LOGIC;--
                        MUSIC: out STD_LOGIC;--
                        SENSOR: in STD_LOGIC;--
                        START : in STD_LOGIC;--
                        LEDS : out STD_LOGIC;
                        SWMUSIC : in STD_LOGIC;--
                        WIN : out STD_LOGIC;--
                        pause : in STD_LOGIC
);
end TOP;

architecture Behavioral of TOP is
-----------------------COMPONENTES-----------------------------
component PmodACL_Demo Port ( CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        SW : in  STD_LOGIC_VECTOR (1 downto 0);
        SDI : in  STD_LOGIC;
        SDO : out  STD_LOGIC;
        SCLK : out  STD_LOGIC;
        SS : out  STD_LOGIC;
        AN : out  STD_LOGIC_VECTOR (7 downto 0);
        SEG : out  STD_LOGIC_VECTOR (6 downto 0);
        DOT : out  STD_LOGIC;--punto
        LED : out  STD_LOGIC_VECTOR (2 downto 0);
                        datosx : out STD_logic_vector (15 downto 0);
                        datosy : out STD_logic_vector (15 downto 0)
                        );
end component;

component texto_fijo port(
        --Entradas
        clk: in std_logic; --Reloj interno de la FPGA
        reset: in std_logic;
        asciiU: in std_logic_vector (7 downto 0); --Vector de 6 bits para introducir ascii desde
switches
        asciiD: in std_logic_vector (7 downto 0); --Vector de 6 bits para introducir ascii desde
switches
        asciiC: in std_logic_vector (7 downto 0); --Vector de 6 bits para introducir ascii desde
switches

        --Salidas
        rs: out std_logic;
        rw: out std_logic;
        enable: out std_logic;
        data: out std_logic_vector(7 downto 0)
);
end component;

component contador_segundos_minutos port (
```

```
        clk_50mhz : in std_logic;
        reset : in std_logic;
        enable : in std_logic;
        segundos_out : out STD_LOGIC_VECTOR(5 downto 0);
                    minutos_out : out STD_LOGIC_VECTOR(5 downto 0)
    );
end component;

component NumberToASCII port (
        number : in std_logic_vector(3 downto 0);
        ascii  : out std_logic_vector(7 downto 0)
    );
end component;


component shift_add3 Port ( num1 : in  STD_LOGIC_VECTOR (5 downto 0);
           UNI : out  STD_LOGIC_VECTOR (3 downto 0);
           DEC : out  STD_LOGIC_VECTOR (3 downto 0);
           CEN : out  STD_LOGIC_VECTOR (3 downto 0);
           MIL : out  STD_LOGIC_VECTOR (3 downto 0));
end component;

component Npwm port (
    clk : in std_logic;        -- clock
    posicion : in std_logic_vector(15 downto 0);  -- reset
    servo : out std_logic
  );
end component;

component pwmp port (
    clk : in std_logic;        -- clock
    posicion : in std_logic_vector(15 downto 0);  -- reset
    servo : out std_logic
  );
end component;

-------------------------SEÑALES--------------------------------

signal x : STD_LOGIC_VECTOR(15 downto 0);
signal y : STD_LOGIC_VECTOR(15 downto 0);

signal segundos: STD_LOGIC_VECTOR(5 dowNTO 0);
signal minutos: STD_LOGIC_VECTOR(5 downto 0);
signal Useg: STD_LOGIC_VECTOR(3 downto 0);
signal Dseg: STD_LOGIC_VECTOR(3 downto 0);
signal Umin: STD_LOGIC_VECTOR(3 downto 0);
signal Dmin: STD_LOGIC_VECTOR(3 downto 0);
signal USascii: std_logic_vector(7 downto 0);
signal DSascii: std_logic_vector(7 downto 0);
signal UMascii: std_logic_vector(7 downto 0);
signal DMascii: std_logic_vector(7 downto 0);
```

```
begin

-- Instancia del componente PmodACL_Demo
   controlACL : PmodACL_Demo
   port map (
      CLK => CLK,
      RST => RST,
      SW => SW,
      SDI => SDI,
      SDO => SDO,
      SCLK => SCLK,
      SS => SS,
      AN => AN,
      SEG => SEG,
      DOT => DOT,
      LED => LED,
                datosx => x,
                datosy =>y

   );

         LCD : texto_fijo
         port map(
        --Entradas
        clk => clk, --Reloj interno de la FPGA
        reset => RST,
        asciiU => USascii,
        asciiD => DSascii,
        asciiC => UMascii,

        --Salidas
        rs => RS,
        rw => RW,
        enable => E,
        data=> DATA
);

        Segundero : contador_segundos_minutos
         port map(
        --Entradas
        clk_50mhz => clk, --Reloj interno de la FPGA
        reset => RST,
        enable =>START,
        --Salidas
        segundos_out => segundos,
        minutos_out => minutos
);

  ShiftSeg : shift_add3
  port map (
        num1 => segundos,
```

```
        UNI => Useg,
        DEC => Dseg
);

ShiftMin : shift_add3
port map (
        num1 => minutos,
        UNI => Umin,
        DEC => Dmin
);

Usegundos : NumberToASCII
port map(
        number => Useg,
        ascii => USascii
);

Dsegundos : NumberToASCII
port map(
        number => Dseg,
        ascii => DSascii
);

Uminutos : NumberToASCII
port map(
        number => Umin,
        ascii => UMascii
);

Dminutos : NumberToASCII
port map(
        number => Dmin,
        ascii => DMascii
);

        pwmx : Npwm
        port map(
        clk => CLK,
        posicion => x,
        servo => servo1
        );

                pwmy : Npwm
        port map(
        clk => CLK,
        posicion => y,
        servo => servo2
        );

        pwmpx : PWMP
        port map(
```

```
        clk => CLK,
        posicion => x,
        servo => servo3
        );

        pwmpy : PWMP
        port map(
        clk => CLK,
        posicion => y,
        servo => servo4
        );

end Behavioral;
```

Los siguientes códigos son los proporcionados por Digilent para el control del dispositivo (PmodACL)

```
--//////////////////////////////////////////////////////////////////////////////////
-- Company: Digilent Inc.
-- Engineer: Josh Sackos
--
-- Create Date:    07/26/2012
-- Module Name:    selData
-- Project Name:        PmodACL_Demo
-- Target Devices: Nexys3
-- Tool versions:  ISE 14.1
-- Description: This module converts axis data from 2's compliment to its magnitude
--                                        representation.  Based upon the input switches SW(1) and
SW(0) either
--                                        the x-axis, y-axis, or z-axis data will be output on the
DOUT output.
--
-- Revision History:
--                                                Revision 0.01 - File Created (Josh Sackos)
--//////////////////////////////////////////////////////////////////////////////////
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use IEEE.std_logic_arith.all;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity sel_Data is
    Port ( CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
                        SW : in STD_LOGIC_VECTOR(1 downto 0);
        xAxis : in  STD_LOGIC_VECTOR (9 downto 0);
        yAxis : in  STD_LOGIC_VECTOR (9 downto 0);
        zAxis : in  STD_LOGIC_VECTOR (9 downto 0);
        DOUTx : out  STD_LOGIC_VECTOR (9 downto 0);
        DOUTy : out  STD_LOGIC_VECTOR (9 downto 0);
```

```
        LED : out  STD_LOGIC_VECTOR (2 downto 0));
end sel_Data;
architecture Behavioral of sel_Data is

begin
-----------------------------------------------
--      Select Display Data and Convert to Magnitude
-----------------------------------------------
process(CLK, RST) begin
                if(RST = '1') then
                                LED <= "000";
                                DOUTx <= "0000000000";
                                DOUTy <= "0000000000";
                elsif rising_edge(CLK) then
                        if(SW = "00") then--ejex
                                LED <= "001";
                        if(xAxis(9) = '1') then
                                DOUTx <= xAxis(9) & ("000000000" - xAxis(8 downto 0));
                        else
                                DOUTx <= xAxis;
                        end if;
--                      elsif(SW = "01") then==eje y
                                LED <= "010";
                        if(yAxis(9) = '1') then
                                DOUTy <= yAxis(9) & ("000000000" - yAxis(8 downto 0));
                        else
                                DOUTy <= yAxis;
                        end if;
--                      elsif(SW = "10") then
--                              LED <= "100";
--                      if(zAxis(9) = '1') then
--                              DOUTz <= zAxis(9) & ("000000000" - zAxis(8 downto 0));
--                      else
--                              DOUTz <= zAxis;
--                      end if;
                        else
                                LED <= "001";
                        if(xAxis(9) = '1') then
                                DOUTx <= xAxis(9) & ("000000000" - xAxis(8 downto 0));
                        else
                                DOUTx <= xAxis;
                        end if;
                        end if;
                        end if;
                        end process;
end Behavioral;


--//////////////////////////////////////////////////////////////////////////////
-- Company: Digilent Inc.
-- Engineer: Andrew Skreen
```

```
--                                  Josh Sackos
--
-- Create Date:    07/26/2012
-- Module Name:    SPIcomponent
-- Project Name:        PmodACL_Demo
-- Target Devices: Nexys3
-- Tool versions:  ISE 14.1
-- Description: The spi_master controls the state of the entire interface. Using
--                                 several signals to interact with the other modules. The spi_master
--                                 selects the data to be transmitted and stores all data received
--                                 from the PmodACL.  The data is then made available to the rest of
--                                         the design on the xAxis, yAxis, and zAxis outputs.
--
--  Inputs:
--              CLK                             100MHz onboard system clock
--              RST                             Main Reset Controller
--              START                                   Signal to initialize a data transfer
--              SDI                             Serial Data In
--
--  Outputs:
--              SDO                             Serial Data Out
--              SCLK                            Serial Clock
--              SS                                      Slave Select
--              xAxis                           x-axis data received from PmodACL
--              yAxis                           y-axis data received from PmodACL
--              zAxis                           z-axis data received from PmodACL
--
-- Revision History:
--                      Revision 0.01 - File Created (Andrew Skreen)
--                      Revision 1.00 - Added comments and modified code (Josh Sackos)
--////////////////////////////////////////////////////////////////////////////////
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
--
=================================================================
================
--                                                      Define Module, Inputs and Outputs
--
=================================================================
================
entity SPIcomponent is
   Port ( CLK : in  STD_LOGIC;
                           RST : in  STD_LOGIC;
                           START : in STD_LOGIC;
          SDI : in  STD_LOGIC;
          SDO : out  STD_LOGIC;
          SCLK : out  STD_LOGIC;
          SS : out  STD_LOGIC;
                           xAxis : out STD_LOGIC_VECTOR(9 downto 0);
                           yAxis : out STD_LOGIC_VECTOR(9 downto 0);
                           zAxis : out STD_LOGIC_VECTOR(9 downto 0)
```

```
        );
end SPIcomponent;

architecture Behavioral of SPIcomponent is

--
==================================================================
=================
--                                                    Component Declarations
--
==================================================================
=================

-- ****************
--                    Serial Port Interface RxTx
-- ****************
component SPIinterface
Port ( txbuffer : in  STD_LOGIC_VECTOR ( 15 downto 0 );
        rxbuffer : out  STD_LOGIC_VECTOR ( 7 downto 0 );
        transmit : in STD_LOGIC;
        done_out : out STD_LOGIC;
        sdi : in STD_LOGIC;
        sdo : out STD_LOGIC;
        rst : in STD_LOGIC;
        clk : in STD_LOGIC;
        sclk : out  STD_LOGIC);
  end component;

-- ****************
--      Serial Port Interface Master Controller
-- ****************
component SPImaster
Port (  rst : in  STD_LOGIC;
        start : in  STD_LOGIC;
        transmit : out  STD_LOGIC;
        done : in STD_LOGIC;
        clk : in STD_LOGIC;
        txdata : out STD_LOGIC_VECTOR ( 15 downto 0 );
        rxdata : in STD_LOGIC_VECTOR ( 7 downto 0 );
        x_axis_data : out STD_LOGIC_VECTOR ( 9 downto 0 );
        y_axis_data : out STD_LOGIC_VECTOR ( 9 downto 0 );
        z_axis_data : out STD_LOGIC_VECTOR ( 9 downto 0 )
                                        );
                end component;

-- ****************
--                        Slave Select Controller
-- ****************
component slaveSelect
Port (  ss : out  STD_LOGIC;
        clk : in STD_LOGIC;
```

```
        done : in STD_LOGIC;
        transmit : in STD_LOGIC;
        rst : in STD_LOGIC);
  end component;
--
=================================================================
=================
--                                                  Signals and Constants
--
=================================================================
=================
        signal TxBuffer : std_logic_vector ( 15 downto 0 );
        signal RxBuffer : std_logic_vector ( 7 downto 0 );
        signal doneConfigure : std_logic;
        signal done : std_logic;
        signal transmit : std_logic;
--
=================================================================
================
--
        Implementation
--
=================================================================
================
begin
-----------------------------------------------------------------------------
--      Controls SPI Interface, Stores Received Data, and Controls Data to Send
-----------------------------------------------------------------------------
C0: SPImaster port map ( RST => RST,
        START => START,
        CLK => CLK,
        transmit => transmit,
        TxData => txBuffer,
        RxData => RxBuffer,
        done => done,
        x_axis_data => xAxis,
        y_axis_data => yAxis,
        z_axis_data => zAxis
    );
-----------------------------------------------------------------------------
--              Produces Timing Signal, Reads ACL Data, and Writes Data to ACL
-----------------------------------------------------------------------------
C1: SPIinterface port map ( SDI => SDI,
        SDO => SDO,
        RST => RST,
        CLK => CLK,
        SCLK => SCLK,
        TxBuffer => TxBuffer,
        RxBuffer => RxBuffer,
        done_out => done,
        transmit => transmit
```

```
   );
-----------------------------------------------------------------------------
--                                                  Enables/Disables PmodACL Communication
-----------------------------------------------------------------------------
C2: slaveSelect port map ( clk => CLK,
         SS => SS,
         done => done,
         transmit => transmit,
         RST => RST
   );
end Behavioral;
}


--////////////////////////////////////////////////////////////////////////////////
-- Company: Digilent Inc.
-- Engineer: Andrew Skreen
--                                  Josh Sackos
--
-- Create Date:    07/26/2012
-- Module Name:    SPImaster
-- Project Name:        PmodACL_Demo
-- Target Devices: Nexys3
-- Tool versions:  ISE 14.1
-- Description: The spi_master controls the state of the entire interface. Using
--several signals to interact with the other modules. The spi_master
--selects the data to be transmitted and stores all data received
--from the PmodACL.  The data is then made available to the rest of
--the design on the xAxis, yAxis, and zAxis outputs.
--
--  Inputs:
--                rst                                     Device Reset Signal
--                clk                                     Base system clock of 100 MHz
--                start                                   start tied to external user input
--                done                    Signal from SPIinterface for completed transmission
--                rxdata                                          Recieved data
--
--  Outputs:
--                transmit                Signal to SPIinterface for beginning of transmission
--                txdata                                  Data out for transmission
--                x_axis_data                             Collected x-Axis data
--                y_axis_data                             Collected y-Axis data
--                z_axis_data                             Collected z-Axis data
--
-- Revision History:
--                        Revision 0.01 - File Created (Andrew Skreen)
--                        Revision 1.00 - Added comments and modified code (Josh Sackos)
--////////////////////////////////////////////////////////////////////////////////
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
```

```
use IEEE.STD_LOGIC_UNSIGNED.ALL;
--
=================================================================
================
--                                                    Define Module, Inputs and Outputs
--
=================================================================
================
entity SPImaster is
   Port ( rst : in  STD_LOGIC;
                          clk : in  STD_LOGIC;
          start : in  STD_LOGIC;
          transmit : out  STD_LOGIC;
                          done : in STD_LOGIC;
                          txdata : out STD_LOGIC_VECTOR ( 15 downto 0 );
                          rxdata : in STD_LOGIC_VECTOR ( 7 downto 0 );
                          x_axis_data : out STD_LOGIC_VECTOR ( 9 downto 0 );
                          y_axis_data : out STD_LOGIC_VECTOR ( 9 downto 0 );
                          z_axis_data : out STD_LOGIC_VECTOR ( 9 downto 0 ));
end SPImaster;

architecture Behavioral of SPImaster is

--
=================================================================
=================
--                                                    Signals and Constants
--
=================================================================
=================

          -- Define FSM states
          type state_type is ( idle , configure , transmitting , recieving , finished , break , holding );
          -- STATE reg
          signal STATE : state_type;

          type data_type is ( x_axis , y_axis , z_axis );
          signal DATA : data_type;

          type configure_type is ( powerCtl , bwRate , dataFormat );
          signal CONFIGUREsel : configure_type;

          --Setting up Configuration Registers
          --POWER_CTL Bits 0x2D
          constant POWER_CTL : std_logic_vector ( 15 downto 0 ) := X"2D08";
          --BW_RATE Bits 0x2C
          constant BW_RATE : std_logic_vector ( 15 downto 0 ) := X"2C08";
          --CONFIG Bits 0x31
          constant DATA_FORMAT : std_logic_vector ( 15 downto 0 ) := X"3100";

          --Axis registers set to only read and do it in single byte increments
```

```
	constant XAXIS0 : std_logic_vector ( 15 downto 0 ) := X"B200"; --10110010;
	constant XAXIS1 : std_logic_vector ( 15 downto 0 ) := X"B300"; --10110011;
	constant YAXIS0 : std_logic_vector ( 15 downto 0 ) := X"B400"; --10110100;
	constant YAXIS1 : std_logic_vector ( 15 downto 0 ) := X"B500"; --10110101;
	constant ZAXIS0 : std_logic_vector ( 15 downto 0 ) := X"B600"; --10110110;
	constant ZAXIS1 : std_logic_vector ( 15 downto 0 ) := X"B700"; --10110111;


	signal break_count : std_logic_vector ( 11 downto 0 );
	signal hold_count : std_logic_vector ( 20 downto 0 );
	signal end_configure : std_logic;
	signal done_configure : std_logic;
	signal register_select : std_logic;
	signal finish : std_logic;
	signal sample_done : std_logic;
	signal prevstart : std_logic_vector (3 downto 0);

--
=====================================================================
================
--
	Implementation
--
=====================================================================
================
begin

spi_masterProcess : process ( clk ) begin

	if rising_edge( clk ) then
	-- Debounce Start Button
		prevstart <= prevstart( 2 downto 0 ) & start;
	--Reset Condition
		if rst = '1' then
			transmit<= '0';
			STATE<= idle;
			DATA<= x_axis;
			break_count<= ( others => '0' );
			hold_count<= ( others => '0' );
			done_configure<= '0';
			CONFIGUREsel<= powerCtl;
			txdata<= ( others => '0' );
			register_select<= '0';
			sample_done<= '0';
			finish<= '0';
			x_axis_data<= ( others => '0' );
			y_axis_data<= ( others => '0' );
			z_axis_data<= ( others => '0' );
			end_configure<= '0';
		else
			--Main State, selects what the overall system is doing
```

```
case STATE is
when idle =>
--If the system has not been configured, enter the configure state
if done_configure = '0' then
	STATE<= configure;
	txdata<= POWER_CTL;
	Transmit<='1';
	--If the system has been configured, enter the transmission state
when start is asserted
elsif prevstart = "0011" and start = '1' and done_configure = '1' then
	STATE<= transmitting;
	finish<='0';
	txdata<= xAxis0;
	sample_done<='0';
end if;
when configure =>
	--Substate of configure selects what configuration is output
	case CONFIGUREsel is
	--Send power control address with desired configuration bits
when powerCtl =>
	STATE<= finished;
	CONFIGUREsel<= bwRate;
	transmit<='1';
	--Send band width rate address with desired configuration bits
when bwRate =>
	txdata<= BW_RATE;
	STATE<= finished;
	CONFIGUREsel<= dataFormat;
	transmit<='1';
	--Send data format address with desired configuration bits
when dataFormat =>
	txdata<= DATA_FORMAT;
	STATE<= finished;
	transmit<='1';
	finish<= '1';
	end_configure<= '1';
when others =>
	null;
end case;

--transmitting leads to the transmission of addresses of data to sample them
when transmitting =>
--Substate of transmitting selects which data register will be sampled
case DATA is
--Selects the x_axis data
	when x_axis =>
	--register_select chooses which of the two registers for each axis
	--will be sampled
		case register_select is
	when '0' =>
	--in each case for data and register_select the state goes to
```

```
--recieving to accept data from SPIinterface and transmit goes
--high to begin the transmission of data to the ACL
STATE<= recieving;
transmit<= '1';
when others =>
STATE<= recieving;
transmit<= '1';
end case;
when y_axis =>
case register_select is
when '0' =>
STATE<= recieving;
transmit<= '1';
when others =>
STATE<= recieving;
transmit<= '1';
end case;
when z_axis =>
case register_select is
when '0' =>
STATE<= recieving;
transmit<= '1';
when others =>
STATE<= recieving;
transmit<= '1';
end case;
when others =>
null;
end case;
--recieving controls the flow of data into the spi_master
when recieving=>
--Substate of Recieving, DATA controls where data will be stored
case DATA is
--Substate of DATA same as in transmitting
when x_axis =>
--register_select controls which half of the output register the
--collected data goes to
case register_select is
when '0' =>
--transmit de-asserted as transmission has already be begun
transmit<= '0';
--when done is asserted, the register to be sampled next
--changes with txdata, data is assigned to the desired output,
--the overall state goes to finish, and register select is
--inverted to go to the other half. this same purpose is used
--throughout DATA in recieving
if done = '1' then
txdata<= xAxis1;
x_axis_data( 7 downto 0 )<= rxdata( 7
downto 0 );
STATE<= finished;
```

```
                                        register_select<='1';
                                end if;
                        when others =>
                                transmit<= '0';
                                if done = '1' then
                                        txdata<= yAxis0;
                                        x_axis_data( 9 downto 8 )<= rxdata( 1
downto 0 );
                                        txdata<= yAxis0;
                                        register_select<= '0';
                                        DATA<= y_axis;
                                        STATE<= finished;
                                end if;
                end case;
                when y_axis =>
                        case register_select is
                        when '0' =>
                                transmit<= '0';
                                if done = '1' then
                                        txdata<= yAxis1;
                                        y_axis_data( 7 downto 0 )<= rxdata( 7
downto 0 );
                                        txdata<= yAxis1;
                                        register_select<='1';
                                        STATE<= finished;
                                end if;
                        when others =>
                                transmit<= '0';
                                if done = '1' then
                                        txdata<= zAxis0;
                                        y_axis_data( 9 downto 8 )<= rxdata( 1
downto 0 );
                                        txdata<= zAxis0;
                                        register_select<= '0';
                                        DATA<= z_axis;
                                        STATE<= finished;
                                end if;
        end case;
        when z_axis =>
                case register_select is
                when '0' =>
                        transmit<= '0';
                        if done = '1' then
                                txdata<= zAxis1;
                                z_axis_data( 7 downto 0 )<= rxdata( 7 downto 0 );
                                txdata<= zAxis1;
                                register_select<='1';
                                STATE<= finished;
                        end if;
                when others =>
                        transmit<= '0';
```

```
                        if done = '1' then
                                txdata<= xAxis0;
                                z_axis_data( 9 downto 8 )<= rxdata( 1 downto 0 );
                                txdata<= xAxis0;
                                register_select<= '0';
                                DATA<= x_axis;
                                STATE<= finished;
                                sample_done<= '1';
                        end if;
        end case;
        when others =>
                        null;
end case;
--finished leads to the break state when transmission completed
when finished =>
        transmit<= '0';
        if done = '1' then
                STATE<= break;
        if end_configure = '1' then
                done_configure<='1';
        end if;
        end if;
--the break state keeps an idle state long enough between transmissions
--to satisfy timing requirements. break can be decreased if desired
when break =>
        if      break_count = X"FFF" then
                break_count<= ( others => '0' );
--only exit to idle if start has been de-asserted ( to keep from
--looping transmitting and recieving undesirably ) and finish and
--sample_done are high showing that the desired action has been
--completed
                if ( finish = '1' or sample_done = '1' ) and start = '0' then
                        STATE<= idle;
                        txdata<= xAxis0;
                --if done configure is high, and sample done is low, the reception
                --has not been completed so the state goes back to transmitting
                elsif sample_done = '1' and start = '1' then
                        STATE<= holding;
                        elsif done_configure = '1' and sample_done = '0' then
                                STATE<= transmitting;
                                transmit<= '1';
                --if the system has not finished configuration, then the state loops
                --back to configure
                                elsif done_configure = '0' then
                                        STATE<= configure;
                                end if;
                        else
                                break_count<= break_count + 1;
                end if;
when holding =>
        if hold_count = X"1FFFFF" then
```

```
                        hold_count<= ( others => '0' );
                        STATE<= transmitting;
                        sample_done<= '0';
                elsif start<= '0' then
                        STATE<= idle;
                        hold_count<= ( others => '0' );
                else
                        hold_count<= hold_count + 1;
                end if;
        when others =>
                null;
        end case;
        end if;
        end if;
        end process;
 end Behavioral;


--//////////////////////////////////////////////////////////////////////////////
-- Company: Digilent Inc.
-- Engineer: Andrew Skreen
--                          Josh Sackos
--
-- Create Date:    07/26/2012
-- Module Name:    SPIinterface
-- Project Name:         PmodACL_Demo
-- Target Devices: Nexys3
-- Tool versions:  ISE 14.1
-- Description: This module involves the control of the SPI interface with three
--processes including the reception of data, the transmission of data,
--and finally the production of the serial data clock used in the SPI
--interface.
--
-- Inputs:
--transmit                    Signal from SPImaster
--txbuffer                    Transmission Buffer, holds signal from SPImaster
--rst                                User input RESET
--clk                                100 MHz clock signal
--sdi                                Serial Data In from ACL
--
-- Outputs:
--rxbuffer                    Recieve Buffer, holds signal from sdi
--done_out                    signal that goes to SPImaster signalling end of Transmission
--      sdo                                 Serial Data Out to ACL
--      sclk                                Serial Clock out to ACL
-- Revision History:
--                      Revision 0.01 - File Created (Andrew Skreen)
--                      Revision 1.00 - Added comments and modified code (Josh Sackos)
--//////////////////////////////////////////////////////////////////////////////
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
--
=====================================================================
===============
--                                                    Define Module, Inputs and Outputs
--
=====================================================================
===============
entity SPIinterface is
   Port ( txbuffer : in  STD_LOGIC_VECTOR ( 15 downto 0 );
        rxbuffer : out  STD_LOGIC_VECTOR ( 7 downto 0 );
                              transmit : in STD_LOGIC;
                              done_out : out STD_LOGIC;
                              sdi : in STD_LOGIC;
                              sdo : out STD_LOGIC;
                              rst : in STD_LOGIC;
                              clk : in STD_LOGIC;
        sclk : out  STD_LOGIC);
end SPIinterface;

architecture Behavioral of SPIinterface is

--
=====================================================================
================
--                                                    Signals and Constants
--
=====================================================================
================

constant CLKDIVIDER : std_logic_vector ( 7 downto 0 ) := X"FF"; --leads to sclk of about
98kHz
        signal clk_count : std_logic_vector ( 7 downto 0 );
        signal clk_edge_buffer : std_logic;

        signal sck_previous : std_logic;
        signal sck_buffer : std_logic;

        signal tx_shift_register : STD_LOGIC_VECTOR ( 15 downto 0 );
        signal rx_shift_register : STD_LOGIC_VECTOR ( 7 downto 0 );

        signal tx_count : std_logic_vector ( 3 downto 0 );
        signal rx_count : std_logic_vector ( 3 downto 0 );
        signal done : std_logic;
        type TxType is ( idle , transmitting );
        signal TxSTATE : TxType;

        type RxType is ( idle , recieving );
        signal RxSTATE : RxType;
```

```
	type SCLKType is ( idle , running );
	signal SCLKSTATE : SCLKType;
--
=====================================================================
===============
--
	Implementation
--
=====================================================================
===============
begin

--Transmission controller
	TxProcess : process ( clk ) begin
	--Reset state
	if rising_edge( clk ) then
		if rst = '1' then
			tx_shift_register<= ( others => '0' );
			tx_count<= ( others => '0' );
			sdo<= '1';
			TxSTATE<= idle;
		else
			case TxSTATE is
			--when idle, if transmit goes high, then the state goes into
			--transmitting. during idle state, sdo is held high
				when idle =>
					tx_shift_register<= txbuffer;
					--sdo<='1';
					if transmit = '1' then
						TxSTATE<= transmitting;
					elsif done = '1' then
						sdo<= '1';
					end if;
				when transmitting =>
					if sck_previous = '1' and sck_buffer = '0' then
					--when count is 15, then cycles out to idle state again.
					--otherwise, the TxData is shifted out on the falling edge
					--of the serial clock
						if tx_count = "1111" then
							TxSTATE<= idle;
							tx_count<= ( others => '0' );
							sdo<= tx_shift_register( 15 );
						else
							tx_count<= tx_count + "0001";
							sdo<= tx_shift_register( 15 );
							tx_shift_register<= tx_shift_register( 14
downto 0 ) & '0';
						end if;
						end if;
				when others =>
					null;
```

```
                end case;
                end if;
                end if;
        end process;

--Reception Controller
RxProcess : process ( clk ) begin
--Reset state
if rising_edge( clk ) then
        if rst = '1' then
                rx_shift_register<= ( others => '0' );
                rx_count<= ( others => '0' );
                done<= '0';
        else
        case RxSTATE is
                when idle =>
                        --when transmit goes high from the SPImaster, the state
                        --goes to recieving and rx_shift_register is zeroed
                        if transmit = '1' then
                                RxSTATE<= recieving;
                                rx_shift_register<= ( others => '0' );
                        elsif SCLKSTATE = idle then
                                done<= '0';
                        end if;
                when recieving =>
                        if sck_previous = '0' and sck_buffer = '1' then
                        --sdi is sampled on the rising edge and after the 16 rising edge,
                        --the state goes back to idle and done is asserted
                                if rx_count = "1111" then
                                        RxSTATE<= idle;
                                        rx_count<= ( others => '0' );
                                        rx_shift_register<= rx_shift_register ( 6 downto 0 ) & sdi;
                                        done<='1';
                                else
                                        rx_count<= rx_count + "0001";
                                        rx_shift_register<= rx_shift_register ( 6 downto 0 ) & sdi;
                                end if;
                        end if;
                when others =>
                        null;
                end case;
                end if;
                end if;
                end process;

--The rxbuffer is tied to the rx_shift_register and is ouput to the SPImaster
rxbuffer<= rx_shift_register;
--sclk generating process
SCLKgen : process ( clk ) begin
--Reset State with SCK held high
        if rising_edge( clk ) then
```

```
		if rst = '1' then
			clk_count<= ( others => '0' );
			SCLKSTATE<= idle;
			sck_previous<= '1';
			sck_buffer<='1';
		else
		case SCLKSTATE is
			when idle =>
				sck_previous<= '1';
				sck_buffer<='1';
				--when transmit is high, the state goes to running
				if transmit = '1' then
					SCLKSTATE<= running;
					clk_count<= ( others => '0' );
					clk_edge_buffer<= '0';
					sck_previous<= '1';
					sck_buffer<='1';
				end if;
			when running =>
				--when done is high, the state goes back to idle
				if done = '1' then
					SCLKSTATE<= idle;
					--if done is nto asserted, the clock continues to be
					--generated
				elsif clk_count = CLKDIVIDER then
					if clk_edge_buffer = '0' then
						sck_buffer<= '1';
						clk_edge_buffer<= '1';
					else
						sck_buffer<= not sck_buffer;
						clk_count<= ( others => '0' );
					end if;
				else
					sck_previous<= sck_buffer;
					clk_count<= clk_count + 1;
				end if;
			when others =>
				null;
		end case;
	end if;
	end if;
end process;

--the signal SCK is tied to the output sclk
		sclk<= sck_buffer;
		--done_out, to the SPImaster, is tied to the signal done produced in the recieving
state machine
		done_out<= done;
end Behavioral;
```

```
--////////////////////////////////////////////////////////////////////////////////
-- Company: Digilent Inc.
-- Engineer: Andrew Skreen
--                                Josh Sackos
--
-- Create Date:    07/26/2012
-- Module Name:    slaveSelect
-- Project Name:         PmodACL_Demo
-- Target Devices: Nexys3
-- Tool versions:  ISE 14.1
-- Description: A simple module involving the switching of the slave select line
--                                        during transmission and idle states.
--
--  Inputs:
--                rst                                   User input Reset
--                transmit                        signal from SPImaster causing ss line to go low
--                                                              ( enable )
--                done                            signal from SPIinterface causing ss line to go
--                                                              high ( disable )
--
--  Outputs:
--                ss                                    ss output to ACL
--
-- Revision History:
--                        Revision 0.01 - File Created (Andrew Skreen)
--                        Revision 1.00 - Added comments and modified code (Josh Sackos)
--////////////////////////////////////////////////////////////////////////////////
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

--
=================================================================
===============
--                                                              Define Module, Inputs and Outputs
--
=================================================================
===============
entity slaveSelect is
   Port (  rst : in STD_LOGIC;
                                clk : in STD_LOGIC;
                                transmit : in STD_LOGIC;
                                done : in STD_LOGIC;
                                ss : out  STD_LOGIC);
end slaveSelect;

architecture Behavioral of slaveSelect is

--
=================================================================
===============
```

```
--
        Implementation
--
================================================================
===============
begin
                ssprocess : process ( clk ) begin
                        if rising_edge( clk ) then
                                --reset state, ss goes high ( disabled )
                                if rst = '1' then
                                        ss<= '1';
                                --if transmitting, then ss goes low ( enabled )
                                elsif transmit = '1' then
                                        ss<= '0';
                                --if done, then ss goes high ( disabled )
                                elsif done = '1' then
                                        ss<= '1';
                                end if;
                        end if;
                end process;

end Behavioral;


library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use IEEE.std_logic_arith.all;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
--
================================================================
===============
--                                                      Define Module, Inputs and Outputs
--
================================================================
===============
entity ssdCtrl is
   Port ( CLK : in  STD_LOGIC;
        RST : in  STD_LOGIC;
        DINx : in  STD_LOGIC_VECTOR (9 downto 0);
                        DINy : in  STD_LOGIC_VECTOR (9 downto 0);
        AN : out  STD_LOGIC_VECTOR (7 downto 0);
        SEG : out  STD_LOGIC_VECTOR (6 downto 0);
                        DOT : out STD_LOGIC;
                        bcdx: out STD_LOGIC_VECTOR(15 downto 0);
                        bcdy: out STD_LOGIC_VECTOR(15 downto 0)

          );
end ssdCtrl;

architecture Behavioral of ssdCtrl is
```

```
--
================================================================
===============
--                                                          Components
--
================================================================
===============

                        component Format_Data Port (
                                        CLK : in  STD_LOGIC;
                                        DCLK : in STD_LOGIC;
                                        RST : in  STD_LOGIC;
                                        DIN : in  STD_LOGIC_VECTOR (8 downto 0);
                                        BCDOUT : out  STD_LOGIC_VECTOR (11 downto 0)
                        );
                        end component;

--
================================================================
===============
--                                                  Signals and Constants
--
================================================================
===============

                        -- 1 kHz Clock Divider
                        constant cntEndVal : STD_LOGIC_VECTOR(15 downto 0) := X"C350";
                        signal clkCount : STD_LOGIC_VECTOR(15 downto 0) := X"0000";
                        signal DCLK : STD_LOGIC := '0';

                        -- 2 Bit Counter
                        signal CNT : STD_LOGIC_VECTOR(2 downto 0) := "000";

                        -- Binary Data to BCD "g" value format x.xx
                        signal bcdDatax : STD_LOGIC_VECTOR(15 downto 0) := X"0000";
                        signal bcdDatay : STD_LOGIC_VECTOR(15 downto 0) := X"0000";

                        -- Output Data Mux
                        signal muxData : STD_LOGIC_VECTOR(3 downto 0);

--
================================================================
===============
--
        Implementation
--
================================================================
===============
begin
                -- Assign minus sign if applicable
                bcdDatax(15 downto 12) <= (X"A") when DINx(9) = '0' else (X"F");
```

```
        bcdDatay(15 downto 12) <= (X"A") when DINy(9) = '0' else (X"F");
        -- Assign DOT when count is 2
--      DOT <= '1' when (CNT = "0100" or CNT ="0111") else '1';
        DOT <= '1';
        BCDX<=bcdDatax;
        BCDY<=bcdDatay;
        -------------------------------
        --                      Format Data
        -------------------------------
        FDATAx : Format_Data port map (
                                CLK => CLK,
                                DCLK => DCLK,
                                RST => RST,
                                DIN => DINx(8 downto 0),
                                BCDOUT => bcdDatax(11 downto 0)
        );
        FDATAy : Format_Data port map (
                                CLK => CLK,
                                DCLK => DCLK,
                                RST => RST,
                                DIN => DINy(8 downto 0),
                                BCDOUT => bcdDatay(11 downto 0)
        );
        ------------------------------------------------
        --                                      Output Data Mux
        --              Select data to display on SSD
        ------------------------------------------------
        process(CNT(1), CNT(0), bcdDatax,bcdDatay, RST) begin
                if(RST = '1') then
                        muxData <= "0000";
                else
                case (CNT) is
                        when "000" => muxData <= bcdDatax(3 downto 0);--unidad
                        when "001" => muxData <= bcdDatax(7 downto 4);--decena
                        when "010" => muxData <= bcdDatax(11 downto 8);--centena
                        when "011" => muxData <= bcdDatax(15 downto 12);--signo
--                      when "100" => muxData <= bcdDatay(3 downto 0);
--                      when "101" => muxData <= bcdDatay(7 downto 4);
--                      when "110" => muxData <= bcdDatay(11 downto 8);
--                      when "111" => muxData <= bcdDatay(15 downto 12);
                        when "100" => muxData <= bcdDatay(15 downto 12);
                        when "101" => muxData <= bcdDatay(11 downto 8);
                        when "110" => muxData <= bcdDatay(7 downto 4);
                        when "111" => muxData <= bcdDatay(3 downto 0);
                        when others => muxData <= "0000";
                end case;
                end if;
        end process;

                --------------------------------
                --                      Segment Decoder
```

```
-- Determines cathode pattern
--   to display digit on SSD
--------------------------------
process(DCLK, RST) begin
        if(RST = '1') then
                        SEG <= "1000000";
        elsif rising_edge(DCLK) then
                case (muxData) is
        --signo(15:12)
        --
                        when X"0" => SEG <= "1000000";  -- 0
                        when X"1" => SEG <= "1111001";  -- 1
                        when X"2" => SEG <= "0100100";  -- 2
                        when X"3" => SEG <= "0110000";  -- 3
                        when X"4" => SEG <= "0011001";  -- 4
                        when X"5" => SEG <= "0010010";  -- 5
                        when X"6" => SEG <= "0000010";  -- 6
                        when X"7" => SEG <= "1111000";  -- 7
                        when X"8" => SEG <= "0000000";  -- 8
                        when X"9" => SEG <= "0010000";  -- 9
                        when X"A" => SEG <= "0111111";  -- Minus
                        when X"F" => SEG <= "1111111";  -- Off
                        when others => SEG <= "1111111";
                end case;
        end if;
end process;

-----------------------------------
--                        Anode Decoder
--    Determines digit digit to
--   illuminate for clock period
-----------------------------------
process(DCLK, RST) begin
        if(RST = '1') then
                        AN <= "11111111";
        elsif rising_edge(DCLK) then
                case (CNT) is
                        when "000" => AN <= "11111110";  -- 0
                        when "001" => AN <= "11111101";  -- 1
                        when "010" => AN <= "11111011";  -- 2
                        when "011" => AN <= "11110111";  -- 3
                        when "100" => AN <= "11101111";  -- 4
                        when "101" => AN <= "11011111";  -- 5
                        when "110" => AN <= "10111111";  -- 6
                        when "111" => AN <= "01111111";  -- 7
                        when others => AN <= "11111111";
                end case;
        end if;
end process;
```

```
--------------------------------
--                    2 Bit Counter
--        Used to select which digit
--         is being illuminated, and
--       selects data to be displayed
--------------------------------
process(DCLK) begin
            if rising_edge(DCLK) then
                        CNT <= CNT + 1;
            end if;
end process;
--------------------------------
--1khz Clock Divider
--  Timing for refreshing the
--                     SSD, etc.
--------------------------------
process(CLK) begin
      if rising_edge(CLK) then
            if(clkCount = cntEndVal) then
                  DCLK <= '1';
                  clkCount <= X"0000";
            else
                  DCLK <= '0';
                  clkCount <= clkCount + 1;
            end if;
      end if;
end process;
end Behavioral;
```

De este código sacamos la posición en grados para os motores, es variable BCDX y BCDY

```
--//////////////////////////////////////////////////////////////////////////////////
-- Company: Digilent Inc.
-- Engineer: Josh Sackos
--
-- Create Date:    07/26/2012
-- Module Name:    Format_Data
-- Project Name:         PmodACL_Demo
-- Target Devices: Nexys3
-- Tool versions:  ISE 14.1
-- Description: The purpose of this module is to calculate the "g" value of the
--                  the input data.  In this demo the accelerometer is configured for
--                   +/- 2g, this means that the input data must be divided by the LSB
--                                  to get the "g" value of the current reading.
--
--                   The calculated/measured "g" value will range from 0.00 to 2.0X. The
--                   calculated "g" value is sent into a binary to BCD converter, and
--                                  the BCD data is output for display on the SSD.
--
```

```
-- Revision History:
--                                                    Revision 0.01 - File Created (Josh Sackos)
--////////////////////////////////////////////////////////////////////////////////
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

--
================================================================================
--                                                                Define Module, Inputs and Outputs
--
================================================================================
entity Format_Data is
   Port ( CLK : in  STD_LOGIC;
                          DCLK : in STD_LOGIC;
          RST : in  STD_LOGIC;
          DIN : in  STD_LOGIC_VECTOR (8 downto 0);
          BCDOUT : out  STD_LOGIC_VECTOR (11 downto 0));
end Format_Data;

architecture Behavioral of Format_Data is

--
================================================================================
--                                                                                    Components
--
================================================================================

                -- ****************
                --                                  1/100 of LSB Division
                -- ****************
                component Div port (
                        clk: in std_logic;
                        rfd: out std_logic;
                        dividend: in std_logic_vector(14 downto 0);
                        divisor: in std_logic_vector(7 downto 0);
                        quotient: out std_logic_vector(14 downto 0);
                        fractional: out std_logic_vector(7 downto 0)
                );
                end component;

                -- ****************
                --                                  Binary to BCD Converter
                -- ****************
                component Binary_To_BCD
                         Port ( CLK : in  STD_LOGIC;
```

```
                                        RST : in  STD_LOGIC;
                                        START : in STD_LOGIC;
                                        BIN : in  STD_LOGIC_VECTOR(8 downto 0);
                                        BCDOUT : inout STD_LOGIC_VECTOR(15 downto 0)
                        );
                end component;
--
=====================================================================
===============
--                                                                      Signals and
Constants
--
=====================================================================
===============

-- Scaled up divisor and scaling factor for "g" calculation to get hundredths place accuracy
        constant DIVISOR : STD_LOGIC_VECTOR(7 downto 0) := "10100011";
        constant SCALING : STD_LOGIC_VECTOR(6 downto 0) := "1000000";
-- Signals for scaled division to determine "g" number
        signal tmpDIVIDEND : STD_LOGIC_VECTOR(15 downto 0);
        signal quo : STD_LOGIC_VECTOR(14 downto 0) := "000000000000000";
        signal rmd : STD_LOGIC_VECTOR(7 downto 0) := "00000000";
        signal rfd : STD_LOGIC;
-- Input/Output data, binary to BCD converter
                        signal inputBCD : STD_LOGIC_VECTOR(8 downto 0) := "000000000";
                        signal outputBCD : STD_LOGIC_VECTOR(15 downto 0) := X"0000";
--
=====================================================================
===============
--
        Implementation
--
=====================================================================
===============
begin

                        -- Calculate scaled up dividend
                        tmpDIVIDEND <= DIN*SCALING;

                        -- Assign input data to binary to BCD converter
                        inputBCD <= quo(8 downto 0);


                        ---------------------------------
                        --                      LSB Division
                        ---------------------------------
                        Division : Div port map (
                                                clk => CLK,
                                                dividend => tmpDIVIDEND(14 downto 0),
                                                divisor => DIVISOR,
                                                rfd => rfd,
```

```
                                        quotient => quo,-- dea aqui sacamos el decimal
                                        fractional => rmd
                        );


                        --------------------------------
                        --                      Binary to BCD
                        --------------------------------
                        BtoBCD : Binary_To_BCD port map (
                                        CLK => CLK,
                                        RST => RST,
                                        START => DCLK,
                                        BIN => inputBCD,
                                        BCDOUT => outputBCD
                        );

                        -- Assign output display data
                        BCDOUT(11 downto 0) <= outputBCD(11 downto 8) & outputBCD(7
downto 4) & outputBCD(3 downto 0);
end Behavioral;


--////////////////////////////////////////////////////////////////////////////////
-- Company: Digilent Inc.
-- Engineer: Josh Sackos
--
-- Create Date:    07/23/2012
-- Module Name:    Binary_To_BCD
-- Project Name:         PmodACL_Demo
-- Target Devices: Nexys3
-- Tool versions:  ISE 14.1
-- Description: This module receives a 9 bit binary value and converts it to
--                                         a packed binary coded decimal (BCD) using the shift add
3
--                                         algorithm.
--
--                                         The output consists of 4 BCD digits as follows:
--
--                                        BCDOUT[15:12]       - Thousands place
--                                        BCDOUT[11:8]        - Hundreds place
--                                        BCDOUT[7:4]                 - Tens place
--                                        BCDOUT[3:0]                 - Ones place
--
-- Revision History:
--                                         Revision 0.01 - File Created (Josh Sackos)
--////////////////////////////////////////////////////////////////////////////////
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use IEEE.std_logic_arith.all;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
```

```
--
==================================================================
===============
--                                                    Define Module, Inputs and Outputs
--
==================================================================
===============
entity Binary_To_BCD is
   Port ( CLK : in  STD_LOGIC;
                  -- 100Mhz CLK
        RST : in  STD_LOGIC;
                  --        Reset
        START : in  STD_LOGIC;
         --        Signal to initialize conversion
        BIN : in  STD_LOGIC_VECTOR (8 downto 0);       --        Binary value to be converted
        BCDOUT : inout  STD_LOGIC_VECTOR (15 downto 0));  --4 digit binary coded
decimal output
end Binary_To_BCD;

architecture Behavioral of Binary_To_BCD is
--
==================================================================
===============
--                                        Signals and Constants
--
==================================================================
===============
                  -- Stores number of shifts executed
                  signal shiftCount : STD_LOGIC_VECTOR(4 downto 0) := "00000";
                  -- Temporary shift regsiter
                  signal tmpSR : STD_LOGIC_VECTOR(27 downto 0) := (others=>'0');
                  -- FSM States
                  type state_type is (Idle, Init, Shift, Check, Done);
                  -- Present state, Next State
                  signal STATE, NSTATE : state_type;
--
==================================================================
===============
--
         Implementation
--
==================================================================
===============
begin

                  --------------------------------
                  --                    State Register
                  --------------------------------
                  STATE_REGISTER: process(CLK, RST) begin
                              if (RST = '1') then
```

```
                                        STATE <= Idle;
                        elsif rising_edge(CLK) then
                                        STATE <= NSTATE;
                        end if;
                end process;


                --------------------------------
                --              Output Logic/Assignment
                --------------------------------
                OUTPUT_LOGIC: process (CLK, RST)
                begin
                        if(RST = '1') then
                        -- Reset/clear values
                                BCDOUT(11 downto 0) <= X"000";
                                tmpSR <= X"0000000";
                        elsif rising_edge(CLK) then
                                case (STATE) is
                                        when Idle =>
                                                BCDOUT <= BCDOUT;
                                                -- Output does not change
                                                tmpSR <= X"0000000";
                                        -- Temp shift reg empty
                                        when Init =>
                                                BCDOUT <= BCDOUT;
                                                -- Output does not change
                                                tmpSR <= "0000000000000000000" & BIN;
                -- Copy input to lower 9 bits
                                        when Shift =>
                                                BCDOUT <= BCDOUT;
                                                -- Output does not change
                                                tmpSR <= tmpSR(26 downto 0) & '0';
                -- Shift left 1 bit
                                                shiftCount <= shiftCount + '1';
                -- Count the shift
                                        when Check =>
                                                BCDOUT <= BCDOUT;
                                                -- Output does not change
-- Not done converting
                                                if(shiftCount /= X"C") then
-- Add 3 to thousands place
                                                        if(tmpSR(27 downto 24) >= X"5") then
                                                                tmpSR(27 downto 24) <=
tmpSR(27 downto 24) + X"3";
                                                        end if;

                        -- Add 3 to hundreds place
                                                        if(tmpSR(23 downto 20) >= X"5") then
                                                                tmpSR(23 downto 20) <=
tmpSR(23 downto 20) + X"3";
                                                        end if;
```

```
                -- Add 3 to tens place
                                if(tmpSR(19 downto 16) >= X"5") then
                                        tmpSR(19 downto 16) <=
tmpSR(19 downto 16) + X"3";
                                end if;
                -- Add 3 to ones place
                                if(tmpSR(15 downto 12) >= X"5") then
                                        tmpSR(15 downto 12) <=
tmpSR(15 downto 12) + X"3";
                                end if;
                        end if;
                when Done =>
                        BCDOUT(11 downto 0) <= tmpSR(23 downto 12);
        -- Assign output the new BCD values
                        tmpSR <= X"0000000";
                        -- Clear temp shift register
                        shiftCount <= "00000";
                -- Clear shift count
            end case;
            end if;
        end process;
        --------------------------------
        --              Next State Logic
        --------------------------------
        NEXT_STATE_LOGIC: process (START, shiftCount, STATE)
        begin
            -- Define default state to avoid latches
            NSTATE <= Idle;
            case (STATE) is
                when Idle =>
                        if(START = '1') then
                                NSTATE <= Init;
                        else
                                NSTATE <= Idle;
                        end if;
                when Init =>
                        NSTATE <= Shift;
                when Shift =>
                        NSTATE <= Check;
                when Check =>
                        if(shiftCount /= X"C") then
                                NSTATE <= Shift;
                        else
                                NSTATE <= Done;
                        end if;
                when Done =>
                        NSTATE <= Idle;
                when others =>
                        NSTATE <= Idle;
            end case;
```

```
                end process;
end Behavioral;


--//////////////////////////////////////////////////////////////////////////////////
-- Company: Digilent Inc.
-- Engineer: Josh Sackos
--
-- Create Date:    07/26/2012
-- Module Name:    ClkDiv_5Hz
-- Project Name:          PmodACL_Demo
-- Target Devices: Nexys3
-- Tool versions:  ISE 14.1
-- Description: Converts input 100MHz clock signal to a 5Hz clock signal.
--
-- Revision History:
--                                                    Revision 0.01 - File Created (Josh Sackos)
--//////////////////////////////////////////////////////////////////////////////////
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use IEEE.std_logic_arith.all;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

--
=====================================================================
================
--                                                                          Define Module
--
=====================================================================
================
entity ClkDiv_5Hz is
   Port ( CLK : in  STD_LOGIC;                          -- 100MHz onboard clock
          RST : in  STD_LOGIC;                          -- Reset
          CLKOUT : inout  STD_LOGIC);             -- New clock output
end ClkDiv_5Hz;

architecture Behavioral of ClkDiv_5Hz is

--
=====================================================================
================
--                                                  Signals and Constants
--
=====================================================================
================

Current count value
          signal clkCount : STD_LOGIC_VECTOR(23 downto 0) := (others => '0');
-- Value to toggle output clock at
          constant cntEndVal : STD_LOGIC_VECTOR(23 downto 0) := X"989680";
```

```
--
=================================================================
===============
--
        Implementation
--
=================================================================
===============
begin

                        ------------------------------------------------
                        --      5Hz Clock Divider Generates Send/Receive signal
                        ------------------------------------------------
                        process(CLK, RST) begin
                                -- Reset clock
                                if(RST = '1')  then
                                        CLKOUT <= '0';
                                        clkCount <= X"000000";
                                elsif rising_edge(CLK) then
                                        if(clkCount = cntEndVal) then
                                                CLKOUT <= NOT CLKOUT;
                                                        clkCount <= X"000000";
                                        else
                                                        clkCount <= clkCount + '1';
                                        end if;
                                end if;
                        end process;
end Behavioral;
```

Este código controla la LCD

```
--//////////////////////////////////////////////////////////////////////////////
-- Company: Digilent Inc.
-- Engineer: Josh Sackos
--
-- Create Date:    07/26/2012
-- Module Name:    ClkDiv_5Hz
-- Project Name:         PmodACL_Demo
-- Target Devices: Nexys3
-- Tool versions:  ISE 14.1
-- Description: Converts input 100MHz clock signal to a 5Hz clock signal.
--
-- Revision History:
--                                                Revision 0.01 - File Created (Josh Sackos)
--//////////////////////////////////////////////////////////////////////////////
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use IEEE.std_logic_arith.all;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
```

```
--
==================================================================
================
--                                                          Define Module
--
==================================================================
================
entity ClkDiv_5Hz is
   Port ( CLK : in  STD_LOGIC;                          -- 100MHz onboard clock
          RST : in  STD_LOGIC;                          -- Reset
          CLKOUT : inout  STD_LOGIC);               -- New clock output
end ClkDiv_5Hz;
architecture Behavioral of ClkDiv_5Hz is
--
==================================================================
================
--                                                  Signals and Constants
--
==================================================================
================
                        -- Current count value
                signal clkCount : STD_LOGIC_VECTOR(23 downto 0) := (others => '0');
                        -- Value to toggle output clock at
                constant cntEndVal : STD_LOGIC_VECTOR(23 downto 0) := X"989680";
--
==================================================================
===============
--
        Implementation
--
==================================================================
===============
begin

                        ------------------------------------------------
                        --      5Hz Clock Divider Generates Send/Receive signal
                        ------------------------------------------------
                        process(CLK, RST) begin

                                -- Reset clock
                                if(RST = '1')  then
                                        CLKOUT <= '0';
                                        clkCount <= X"000000";
                                elsif rising_edge(CLK) then
                                        if(clkCount = cntEndVal) then
                                                CLKOUT <= NOT CLKOUT;
                                                clkCount <= X"000000";
                                        else
                                                clkCount <= clkCount + '1';
                                        end if;
```

```
                                        end if;
                            end process;
end Behavioral;
```

Este código nos proporciona el contador de minutos y segundos

```
--////////////////////////////////////////////////////////////////////////////////
-- Company: Digilent Inc.
-- Engineer: Josh Sackos
--
-- Create Date:    07/26/2012
-- Module Name:    ClkDiv_5Hz
-- Project Name:          PmodACL_Demo
-- Target Devices: Nexys3
-- Tool versions:  ISE 14.1
-- Description: Converts input 100MHz clock signal to a 5Hz clock signal.
--
-- Revision History:
--                                                 Revision 0.01 - File Created (Josh Sackos)
--////////////////////////////////////////////////////////////////////////////////
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use IEEE.std_logic_arith.all;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

--
=====================================================================
================
--                                                                        Define Module
--
=====================================================================
================
entity ClkDiv_5Hz is
   Port ( CLK : in  STD_LOGIC;                          -- 100MHz onboard clock
          RST : in  STD_LOGIC;                          -- Reset
          CLKOUT : inout  STD_LOGIC);              -- New clock output
end ClkDiv_5Hz;

architecture Behavioral of ClkDiv_5Hz is
--
=====================================================================
================
--                                                    Signals and Constants
--
=====================================================================
================

                    -- Current count value
                    signal clkCount : STD_LOGIC_VECTOR(23 downto 0) := (others => '0');
                    -- Value to toggle output clock at
```

```
                constant cntEndVal : STD_LOGIC_VECTOR(23 downto 0) := X"989680";
--
=================================================================
===============
--
        Implementation
--
=================================================================
===============
begin
                        ------------------------------------------------
                        --      5Hz Clock Divider Generates Send/Receive signal
                        ------------------------------------------------
                        process(CLK, RST) begin
                                -- Reset clock
                                if(RST = '1')  then
                                        CLKOUT <= '0';
                                        clkCount <= X"000000";
                                elsif rising_edge(CLK) then
                                        if(clkCount = cntEndVal) then
                                                CLKOUT <= NOT CLKOUT;
                                                clkCount <= X"000000";
                                        else
                                                clkCount <= clkCount + '1';
                                        end if;
                                end if;
                        end process;
end Behavioral;


--//////////////////////////////////////////////////////////////////////////////
-- Company: Digilent Inc.
-- Engineer: Josh Sackos
--
-- Create Date:    07/26/2012
-- Module Name:    ClkDiv_5Hz
-- Project Name:         PmodACL_Demo
-- Target Devices: Nexys3
-- Tool versions:  ISE 14.1
-- Description: Converts input 100MHz clock signal to a 5Hz clock signal.
--
-- Revision History:
--                                                Revision 0.01 - File Created (Josh Sackos)
--//////////////////////////////////////////////////////////////////////////////
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use IEEE.std_logic_arith.all;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

--
```

```
=================================================================
================
--                                                    Define Module
--
=================================================================
================
entity ClkDiv_5Hz is
   Port ( CLK : in  STD_LOGIC;                    -- 100MHz onboard clock
          RST : in  STD_LOGIC;                    -- Reset
          CLKOUT : inout  STD_LOGIC);             -- New clock output
end ClkDiv_5Hz;

architecture Behavioral of ClkDiv_5Hz is

--
=================================================================
================
--                                                Signals and Constants
--
=================================================================
================
                -- Current count value
                signal clkCount : STD_LOGIC_VECTOR(23 downto 0) := (others => '0');
                -- Value to toggle output clock at
                constant cntEndVal : STD_LOGIC_VECTOR(23 downto 0) := X"989680";
--
=================================================================
===============
--
        Implementation
--
=================================================================
===============
begin
                        ------------------------------------------------
                        --      5Hz Clock Divider Generates Send/Receive signal
                        ------------------------------------------------
                        process(CLK, RST) begin

                                -- Reset clock
                                if(RST = '1')  then
                                        CLKOUT <= '0';
                                        clkCount <= X"000000";
                                elsif rising_edge(CLK) then
                                        if(clkCount = cntEndVal) then
                                                CLKOUT <= NOT CLKOUT;
                                                clkCount <= X"000000";
                                        else
                                                clkCount <= clkCount + '1';
                                        end if;
                                end if;
```

```
                    end process;
end Behavioral;
```

Separación de unidades para el segundero

```
library IEEE;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity shift_add3 is
   Port ( num1 : in  STD_LOGIC_VECTOR (5 downto 0);
        UNI : out  STD_LOGIC_VECTOR (3 downto 0);
        DEC : out  STD_LOGIC_VECTOR (3 downto 0);
        CEN : out  STD_LOGIC_VECTOR (3 downto 0);
        MIL : out  STD_LOGIC_VECTOR (3 downto 0));
end shift_add3;

architecture Behavioral of shift_add3 is
signal P: std_logic_vector (15 DOWNTO 0); -- asigna UNI, DEC,CEN, MIL
signal num: std_logic_vector (13 downto 0);
begin
num<= x"00" & num1;
        PROCESS(num)
        VARIABLE UM_C_D_U:STD_LOGIC_VECTOR(29 DOWNTO 0);
        --30 bits para separar las U.Millar-Centenas-Decenas-Unidades
        BEGIN
        --ciclo de inicialización
        FOR I IN 0 TO 29 LOOP --
                UM_C_D_U(I):='0'; -- se inicializa con 0
        END LOOP;

        UM_C_D_U(13 DOWNTO 0):=num(13 downto 0); --contador de 14 bits
        -- UM_C_D_U(17 DOWNTO 4):=CONT(13 downto 0); --contador de 14 bits, carga
desde

        -- el shift4

        --ciclo de asignación UM-C-D-U
        FOR I IN 0 TO 13 LOOP
                -- FOR I IN 0 TO 9 LOOP -- si carga desde shift4 solo hace 10 veces el ciclo
shift add
                -- los siguientes condicionantes comparan (>=5) y suman 3
                IF UM_C_D_U(17 DOWNTO 14) > 4 THEN -- U
                        UM_C_D_U(17 DOWNTO 14):= UM_C_D_U(17 DOWNTO 14)+3;
                END IF;
                IF UM_C_D_U(21 DOWNTO 18) > 4 THEN -- D
                        UM_C_D_U(21 DOWNTO 18):= UM_C_D_U(21 DOWNTO 18)+3;
                END IF;
                IF UM_C_D_U(25 DOWNTO 22) > 4 THEN -- C
                        UM_C_D_U(25 DOWNTO 22):= UM_C_D_U(25 DOWNTO 22)+3;
```

```
                END IF;
                IF UM_C_D_U(29 DOWNTO 26) > 4 THEN -- UM
                        UM_C_D_U(29 DOWNTO 26):= UM_C_D_U(29 DOWNTO 26)+3;
                END IF;
                -- realiza el corrimiento
                UM_C_D_U(29 DOWNTO 1):= UM_C_D_U(28 DOWNTO 0);
        END LOOP;
        P<=UM_C_D_U(29 DOWNTO 14); -- guarda en P y en seguida se separan UM-C-D-U

END PROCESS;

--UNIDADES
UNI<=P(3 DOWNTO 0);
--DECENAS
DEC<=P(7 DOWNTO 4);
--CENTENAS
CEN<=P(11 DOWNTO 8);
--MILLARES
MIL<=P(15 DOWNTO 12);

end Behavioral;


library IEEE;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity shift_add3 is
   Port ( num1 : in  STD_LOGIC_VECTOR (5 downto 0);
        UNI : out  STD_LOGIC_VECTOR (3 downto 0);
        DEC : out  STD_LOGIC_VECTOR (3 downto 0);
        CEN : out  STD_LOGIC_VECTOR (3 downto 0);
        MIL : out  STD_LOGIC_VECTOR (3 downto 0));
end shift_add3;

architecture Behavioral of shift_add3 is
signal P: std_logic_vector (15 DOWNTO 0); -- asigna UNI, DEC,CEN, MIL
signal num: std_logic_vector (13 downto 0);
begin
num<= x"00" & num1;
        PROCESS(num)
        VARIABLE UM_C_D_U:STD_LOGIC_VECTOR(29 DOWNTO 0);
        --30 bits para separar las U.Millar-Centenas-Decenas-Unidades
        BEGIN
        --ciclo de inicialización
        FOR I IN 0 TO 29 LOOP --
                UM_C_D_U(I):='0'; -- se inicializa con 0
        END LOOP;

        UM_C_D_U(13 DOWNTO 0):=num(13 downto 0); --contador de 14 bits
```

```
        -- UM_C_D_U(17 DOWNTO 4):=CONT(13 downto 0); --contador de 14 bits, carga
desde

        -- el shift4

        --ciclo de asignación UM-C-D-U
        FOR I IN 0 TO 13 LOOP
                -- FOR I IN 0 TO 9 LOOP -- si carga desde shift4 solo hace 10 veces el ciclo
shift add
                -- los siguientes condicionantes comparan (>=5) y suman 3
                IF UM_C_D_U(17 DOWNTO 14) > 4 THEN -- U
                        UM_C_D_U(17 DOWNTO 14):= UM_C_D_U(17 DOWNTO 14)+3;
                END IF;
                IF UM_C_D_U(21 DOWNTO 18) > 4 THEN -- D
                        UM_C_D_U(21 DOWNTO 18):= UM_C_D_U(21 DOWNTO 18)+3;
                END IF;
                IF UM_C_D_U(25 DOWNTO 22) > 4 THEN -- C
                        UM_C_D_U(25 DOWNTO 22):= UM_C_D_U(25 DOWNTO 22)+3;
                END IF;
                IF UM_C_D_U(29 DOWNTO 26) > 4 THEN -- UM
                        UM_C_D_U(29 DOWNTO 26):= UM_C_D_U(29 DOWNTO 26)+3;
                END IF;
                -- realiza el corrimiento
                UM_C_D_U(29 DOWNTO 1):= UM_C_D_U(28 DOWNTO 0);
        END LOOP;
        P<=UM_C_D_U(29 DOWNTO 14); -- guarda en P y en seguida se separan UM-C-D-U

END PROCESS;

--UNIDADES
UNI<=P(3 DOWNTO 0);
--DECENAS
DEC<=P(7 DOWNTO 4);
--CENTENAS
CEN<=P(11 DOWNTO 8);
--MILLARES
MIL<=P(15 DOWNTO 12);

end Behavioral;
```

Convierte de binario a ASCII para el LCD

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity NumberToASCII is
   port (
      number : in std_logic_vector(3 downto 0);
      ascii  : out std_logic_vector(7 downto 0)
   );
```

```
end NumberToASCII;

architecture Behavioral of NumberToASCII is
begin
   process(number)
   begin
      case number is
                when x"0"    => ascii <= x"30";  -- ASCII para el número 0 en hexadecimal
                when x"1"    => ascii <= x"31";  -- ASCII para el número 1 en hexadecimal
                when x"2"    => ascii <= x"32";  -- ASCII para el número 2 en hexadecimal
                when x"3"    => ascii <= x"33";  -- ASCII para el número 3 en hexadecimal
                when x"4"    => ascii <= x"34";  -- ASCII para el número 4 en hexadecimal
                when x"5"    => ascii <= x"35";  -- ASCII para el número 5 en hexadecimal
                when x"6"    => ascii <= x"36";  -- ASCII para el número 6 en hexadecimal
                when x"7"    => ascii <= x"37";  -- ASCII para el número 7 en hexadecimal
                when x"8"    => ascii <= x"38";  -- ASCII para el número 8 en hexadecimal
                when x"9"    => ascii <= x"39";  -- ASCII para el número 9 en hexadecimal
         -- otros casos
                when others => ascii <= x"00"; -- manejo de otros casos (opcional)
         end case;
   end process;
end Behavioral;
```

PWM para el control de servos que van en sentido opuesto

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity NumberToASCII is
   port (
      number : in std_logic_vector(3 downto 0);
      ascii  : out std_logic_vector(7 downto 0)
   );
end NumberToASCII;

architecture Behavioral of NumberToASCII is
begin
   process(number)
   begin
      case number is
         when x"0"    => ascii <= x"30";  -- ASCII para el número 0 en hexadecimal
         when x"1"    => ascii <= x"31";  -- ASCII para el número 1 en hexadecimal
         when x"2"    => ascii <= x"32";  -- ASCII para el número 2 en hexadecimal
         when x"3"    => ascii <= x"33";  -- ASCII para el número 3 en hexadecimal
         when x"4"    => ascii <= x"34";  -- ASCII para el número 4 en hexadecimal
         when x"5"    => ascii <= x"35";  -- ASCII para el número 5 en hexadecimal
         when x"6"    => ascii <= x"36";  -- ASCII para el número 6 en hexadecimal
         when x"7"    => ascii <= x"37";  -- ASCII para el número 7 en hexadecimal
         when x"8"    => ascii <= x"38";  -- ASCII para el número 8 en hexadecimal
         when x"9"    => ascii <= x"39";  -- ASCII para el número 9 en hexadecimal
         -- otros casos
```

```
            when others => ascii <= x"00"; -- manejo de otros casos (opcional)
        end case;
      end process;
end Behavioral;


library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity pwmP is
  Generic (
    msb : integer := 8; -- número de bits del contador
    min : integer := 31668; -- valor mínimo del contador para el tiempo en alto
    max : integer := 124186; -- valor máximo del contador para el tiempo en alto
    inc : integer := 2500; -- incremento para el tiempo en alto
    N : integer := 15 -- divisor
  );
  port (
    clk : in std_logic; -- clock
    posicion : in std_logic_vector(15 downto 0); -- reset
    servo : out std_logic
  );
end entity pwmP;

architecture behavioral of pwmP is
  signal contador : unsigned(36 downto 0);
  signal ref : integer range 0 to 36 := 0;
  signal pw : std_logic;

  -- declaración de señales
  signal clkdiv: std_logic_vector(15 downto 0); -- señal para el divisor
  signal cntPWM: integer range 1 to  1000000 := 1; -- contador de 10ms @ clk=50MHz
  signal cnt: integer range 0 to 37 := 0; -- contador de 0 a 35
  signal high: integer range min to max := min; -- duración del tiempo en alto
begin
  servo <= pw;

  process (posicion) is
  begin
    case posicion is
      when "1010000010010000" => ref <= 0; -- 90
      when "1010000010000101" => ref <= 1; -- 85
      when "1010000010000000" => ref <= 2; -- 80
      when "1010000001110101" => ref <= 3; -- 75
      when "1010000001110000" => ref <= 4; -- 70
      when "1010000001100101" => ref <= 5; -- 65
      when "1010000001100000" => ref <= 6; -- 60
      when "1010000001010101" => ref <= 7; -- 55
      when "1010000001010000" => ref <= 8; -- 50
      when "1010000001000101" => ref <= 9; -- 45
```

```
      when "1010000001000000" => ref <= 10; -- 40
      when "1010000000110101" => ref <= 11; -- 35
      when "1010000000110000" => ref <= 12; -- 30
      when "1010000000100101" => ref <= 13; -- 25
      when "1010000000100000" => ref <= 14; -- 20
      when "1010000000010101" => ref <= 15; -- 15
      when "1010000000010000" => ref <= 16; -- 10
      when "1010000000000101" => ref <= 17; -- 5
      when "0000000000000000" => ref <= 18; -- 0 POSITIVOS HACIA ABAJO
      when "1111000000000101" => ref <= 19; -- 5
      when "1111000000010000" => ref <= 20; -- 10
      when "1111000000010101" => ref <= 21; -- 15
      when "1111000000100000" => ref <= 22; -- 20
      when "1111000000100101" => ref <= 23; -- 25
      when "1111000000110000" => ref <= 24; -- 30
      when "1111000000110101" => ref <= 25; -- 35
      when "1111000001000000" => ref <= 26; -- 40
      when "1111000001000101" => ref <= 27; -- 45
      when "1111000001010000" => ref <= 28; -- 50
      when "1111000001010101" => ref <= 29; -- 55
      when "1111000001100000" => ref <= 30; -- 60
      when "1111000001100101" => ref <= 31; -- 65
      when "1111000001110000" => ref <= 32; -- 70
      when "1111000001110101" => ref <= 33; -- 75
      when "1111000010000000" => ref <= 34; -- 80
      when "1111000010000101" => ref <= 35; -- 85
      when "1111000010010000" => ref <= 36; -- 90
      when others => null;
    end case;
  end process;

  -- Sequential Logic
  process (clk) is
  begin
    if rising_edge(clk) then
      cntPWM <= cntPWM + 1; -- contador de 1 a 500,000
      high <= min + (ref*inc);
    end if;

    if cntPWM <= high then
      pw <= '1';
    else
      pw <= '0';
    end if;
  end process;
end architecture behavioral;
```

PWM para el control de servos que van en sentido normal

```
library ieee;
```

```
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity pwmP is
  Generic (
    msb : integer := 8; -- número de bits del contador
    min : integer := 31668; -- valor mínimo del contador para el tiempo en alto
    max : integer := 124186; -- valor máximo del contador para el tiempo en alto
    inc : integer := 2500; -- incremento para el tiempo en alto
    N : integer := 15 -- divisor
  );
  port (
    clk : in std_logic; -- clock
    posicion : in std_logic_vector(15 downto 0); -- reset
    servo : out std_logic
  );
end entity pwmP;

architecture behavioral of pwmP is
  signal contador : unsigned(36 downto 0);
  signal ref : integer range 0 to 36 := 0;
  signal pw : std_logic;

  -- declaración de señales
  signal clkdiv: std_logic_vector(15 downto 0); -- señal para el divisor
  signal cntPWM: integer range 1 to  1000000 := 1; -- contador de 10ms @ clk=50MHz
  signal cnt: integer range 0 to 37 := 0; -- contador de 0 a 35
  signal high: integer range min to max := min; -- duración del tiempo en alto
begin
  servo <= pw;

  process (posicion) is
  begin
    case posicion is
      when "1010000010010000" => ref <= 0; -- 90
      when "1010000010000101" => ref <= 1; -- 85
      when "1010000010000000" => ref <= 2; -- 80
      when "1010000001110101" => ref <= 3; -- 75
      when "1010000001110000" => ref <= 4; -- 70
      when "1010000001100101" => ref <= 5; -- 65
      when "1010000001100000" => ref <= 6; -- 60
      when "1010000001010101" => ref <= 7; -- 55
      when "1010000001010000" => ref <= 8; -- 50
      when "1010000001000101" => ref <= 9; -- 45
      when "1010000001000000" => ref <= 10; -- 40
      when "1010000000110101" => ref <= 11; -- 35
      when "1010000000110000" => ref <= 12; -- 30
      when "1010000000100101" => ref <= 13; -- 25
      when "1010000000100000" => ref <= 14; -- 20
      when "1010000000010101" => ref <= 15; -- 15
      when "1010000000010000" => ref <= 16; -- 10
```

```
      when "1010000000000101" => ref <= 17; -- 5
      when "0000000000000000" => ref <= 18; -- 0 POSITIVOS HACIA ABAJO
      when "1111000000000101" => ref <= 19; -- 5
      when "1111000000010000" => ref <= 20; -- 10
      when "1111000000010101" => ref <= 21; -- 15
      when "1111000000100000" => ref <= 22; -- 20
      when "1111000000100101" => ref <= 23; -- 25
      when "1111000000110000" => ref <= 24; -- 30
      when "1111000000110101" => ref <= 25; -- 35
      when "1111000001000000" => ref <= 26; -- 40
      when "1111000001000101" => ref <= 27; -- 45
      when "1111000001010000" => ref <= 28; -- 50
      when "1111000001010101" => ref <= 29; -- 55
      when "1111000001100000" => ref <= 30; -- 60
      when "1111000001100101" => ref <= 31; -- 65
      when "1111000001110000" => ref <= 32; -- 70
      when "1111000001110101" => ref <= 33; -- 75
      when "1111000010000000" => ref <= 34; -- 80
      when "1111000010000101" => ref <= 35; -- 85
      when "1111000010010000" => ref <= 36; -- 90
      when others => null;
    end case;
  end process;

  -- Sequential Logic
  process (clk) is
  begin
    if rising_edge(clk) then
      cntPWM <= cntPWM + 1; -- contador de 1 a 500,000
      high <= min + (ref*inc);
    end if;

    if cntPWM <= high then
      pw <= '1';
    else
      pw <= '0';
    end if;
  end process;
end architecture behavioral;
```

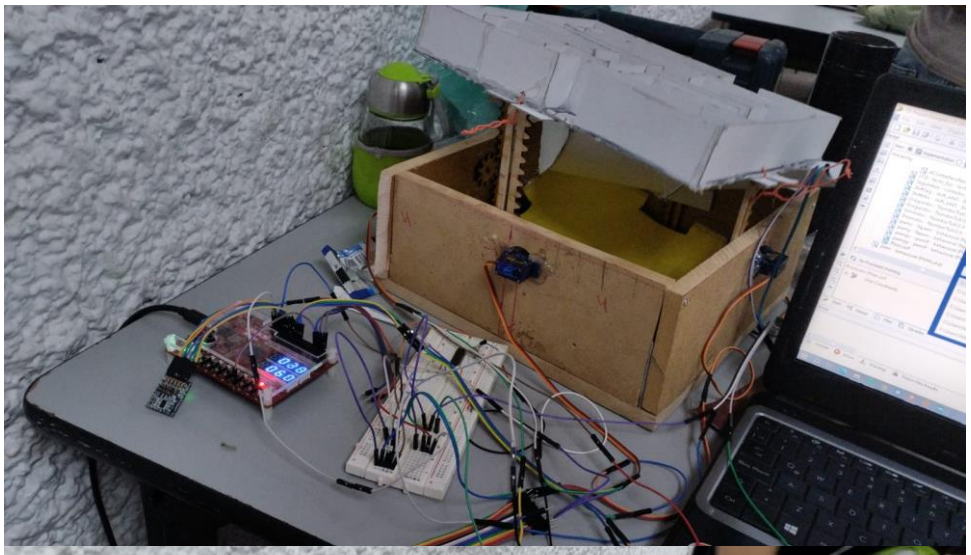
Pines (UCF)

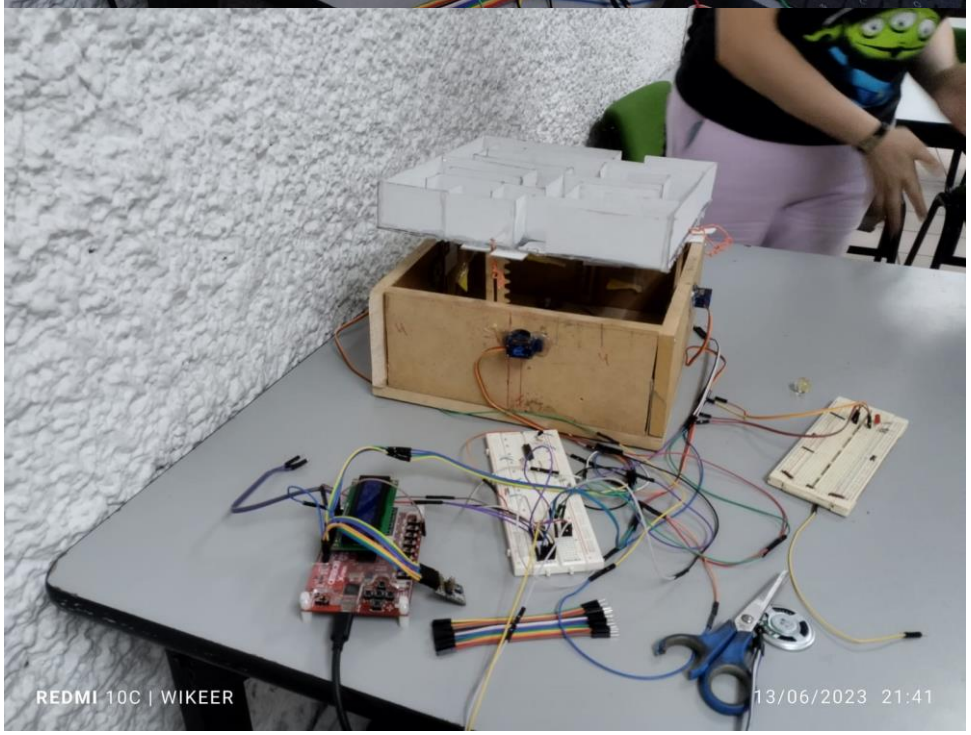
```
# Clock
NET "CLK"    LOC = "E7";

# Buttons
NET "RST"    LOC = "J3";              #BTND

# Switches
NET "SW<0>" LOC = "L1";
NET "SW<1>" LOC = "N2";
```

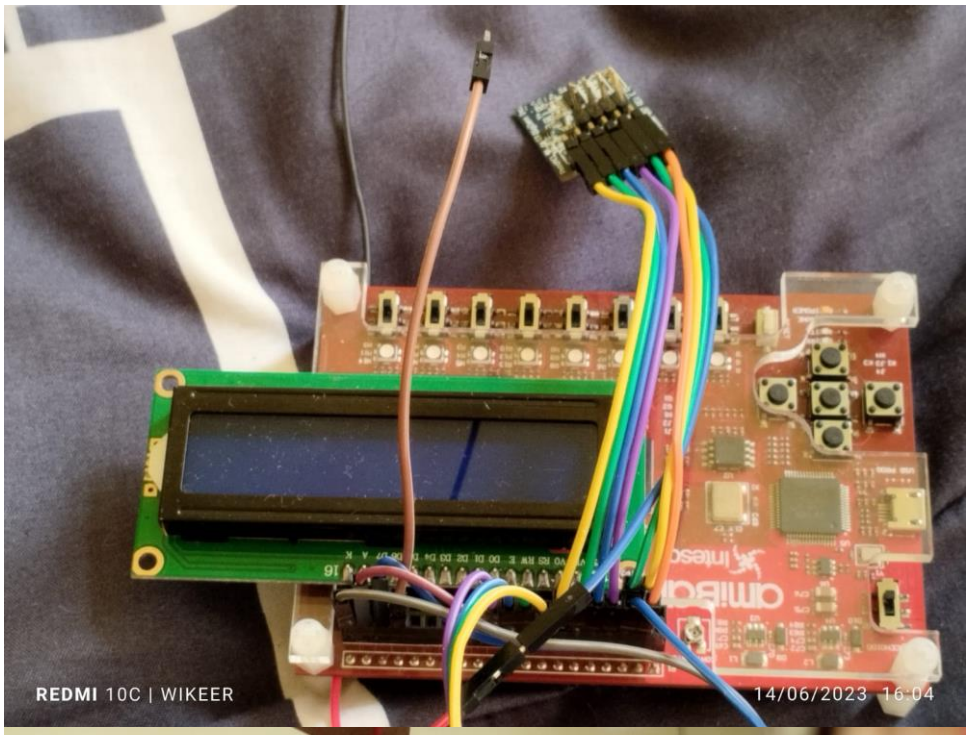
```
#NET "SW<2>" LOC = "V9";

#         ACL Outputs
NET "SS"                LOC = "D10";                    #JB1
NET "SDO"    LOC = "C7";                    #JB2
NET "SDI"     LOC = "D5";                    #JB3
NET "SCLK"  LOC = "E4";                    #JB4

#  Ouput LEDs
NET "LED<0>" LOC = "N14";                  #LD0
NET "LED<1>" LOC = "N7";                   #LD1
NET "LED<2>" LOC = "N5";                   #LD2

#         7 segment display
Net "SEG<0>" LOC = K13 ;// = LVCMOS33; #Bank = 1, pin name = IO_L51P_M1DQ12, Sch
name = CA
Net "SEG<1>" LOC = L15 ;// = LVCMOS33; #Bank = 1, pin name = IO_L51N_M1DQ13, Sch
name = CB
Net "SEG<2>" LOC = M15 ;// = LVCMOS33; #Bank = 1, pin name = IO_L52P_M1DQ14, Sch
name = CC
Net "SEG<3>" LOC = P15 ;// = LVCMOS33; #Bank = 1, pin name = IO_L52N_M1DQ15, Sch
name = CD
Net "SEG<4>" LOC = P14 ;// = LVCMOS33; #Bank = 1, pin name = IO_L53P, Sch name = CE
Net "SEG<5>" LOC = K15 ;// = LVCMOS33; #Bank = 1, pin name = IO_L53N_VREF, Sch
name = CF
Net "SEG<6>" LOC = L14 ;// = LVCMOS33; #Bank = 1, pin name = IO_L61P, Sch name =
CG
Net "DOT" LOC = N15 ;// = LVCMOS33; #Bank = 1, pin name = IO_L61N, Sch name = DP

Net "AN<0>" LOC = "K12" ;// = LVCMOS33; #Bank = 1, pin name = IO_L50N_M1UDQSN,
Sch name = AN0
Net "AN<1>" LOC = "J13" ;// = LVCMOS33; #Bank = 1, pin name = IO_L50P_M1UDQS,
Sch name = AN1
Net "AN<2>" LOC = "K11" ;// = LVCMOS33; #Bank = 1, pin name = IO_L49N_M1DQ11,
Sch name = AN2
Net "AN<3>" LOC = "H13" ;// = LVCMOS33; #Bank = 1, pin name = IO_L49P_M1DQ10,
Sch name = AN3
Net "AN<4>" LOC = "G12" ;// = LVCMOS33; #Bank = 1, pin name = IO_L50N_M1UDQSN,
Sch name = AN4
Net "AN<5>" LOC = "F13" ;// = LVCMOS33; #Bank = 1, pin name = IO_L50P_M1UDQS,
Sch name = AN5
Net "AN<6>" LOC = "H12" ;// = LVCMOS33; #Bank = 1, pin name = IO_L49N_M1DQ11,
Sch name = AN6
Net "AN<7>" LOC = "G13" ;// = LVCMOS33; #Bank = 1, pin name = IO_L49P_M1DQ10,
Sch name = AN7

//Bus de datos
Net "data<0>" LOC = B3 ;// | IOSTANDARD = LVCMOS33; # data1
Net "data<1>" LOC = A5 ;//| IOSTANDARD = LVCMOS33; # data2
Net "data<2>" LOC = C6 ;//| IOSTANDARD = LVCMOS33; # data3
Net "data<3>" LOC = A8 ;//| IOSTANDARD = LVCMOS33; # data4
```

```
Net "data<4>" LOC = C9 ;//| IOSTANDARD = LVCMOS33; # data7
Net "data<5>" LOC = A11 ;//| IOSTANDARD = LVCMOS33; # data8
Net "data<6>" LOC = B13 ;//| IOSTANDARD = LVCMOS33; # data9
Net "data<7>" LOC = B15 ;//| IOSTANDARD = LVCMOS33; # data10
//Pines de control
Net "rs" LOC = D13 ;//| IOSTANDARD = LVCMOS33; # JB7
Net "rw" LOC = E2 ;//| IOSTANDARD = LVCMOS33; # JB8
Net "e" LOC = A2 ;//| IOSTANDARD = LVCMOS33; # JB9
//SERVOS
NET "SERVO1" LOC = "C12" ;//AMARILLO PARALELO 2
NET "SERVO2" LOC = "E1" ;//VERDE
NET "SERVO3" LOC = "C1" ;//AZUL PARALELO 4
NET "SERVO4" LOC = "A3" ;//MORADO
//PINES DE MUSICA
#NET "LOSE" LOC = "E15"; //GRIS
###NET "MUSIC" LOC = "C15"; //MORADO
#NET "WIN" LOC = "B14"; //AZUL
#NET "SWMUSIC" LOC = "M8";
#//CONTROL
#NET "SENSOR" LOC = "A12";//CAFE
NET "START" LOC = "P5";
#NET "LEDS" LOC = "R5";
#NET "PAUSE" LOC = "N6";
```

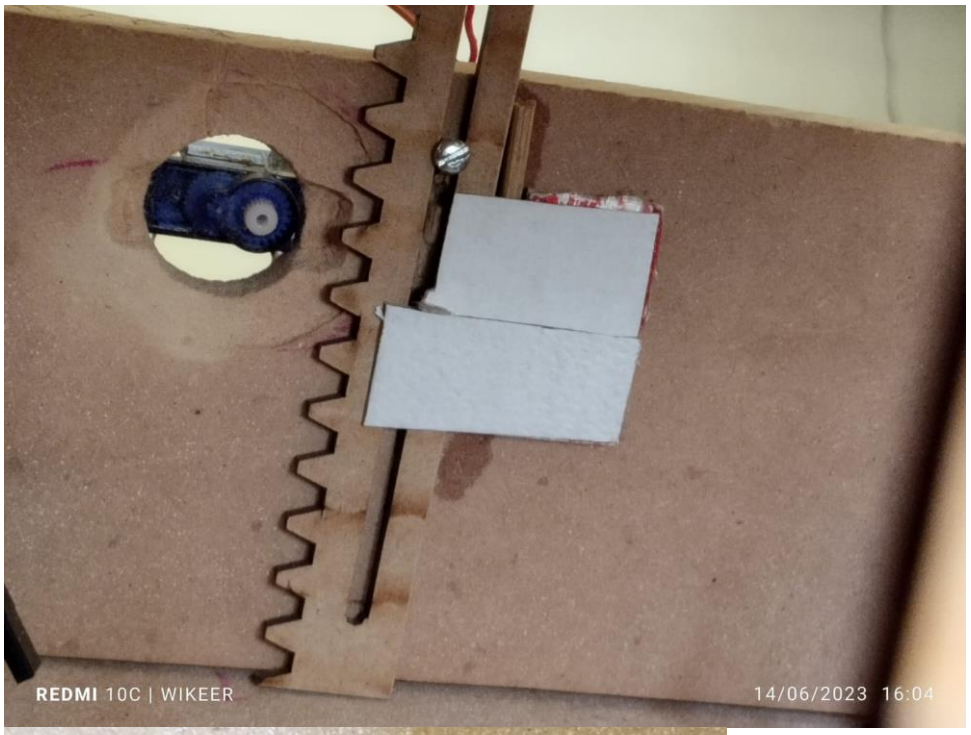
FOTOS



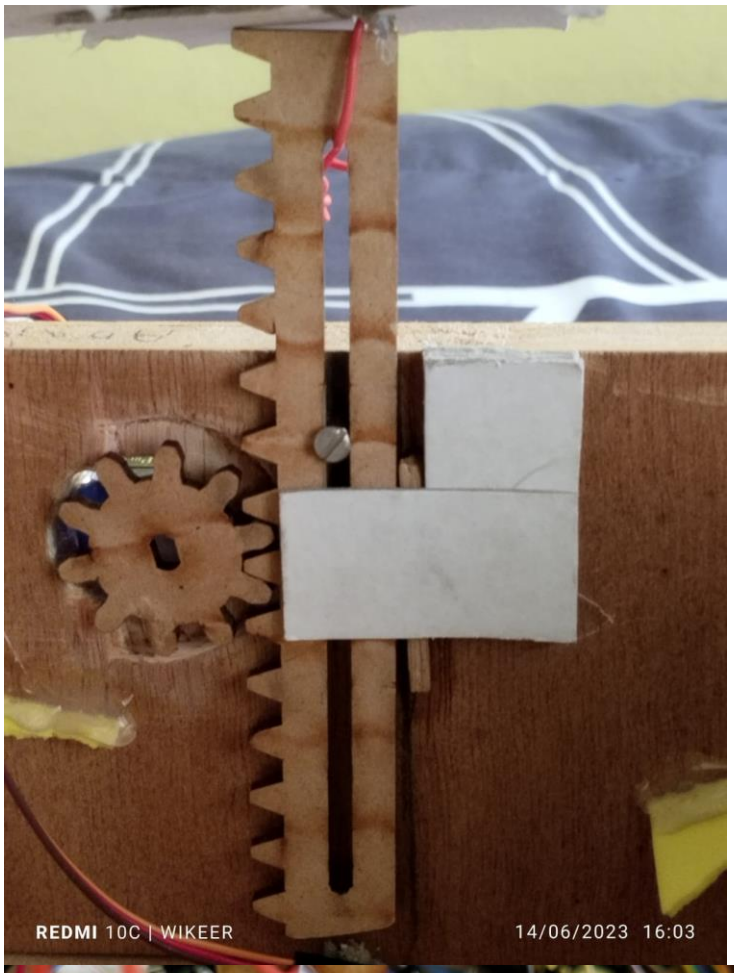


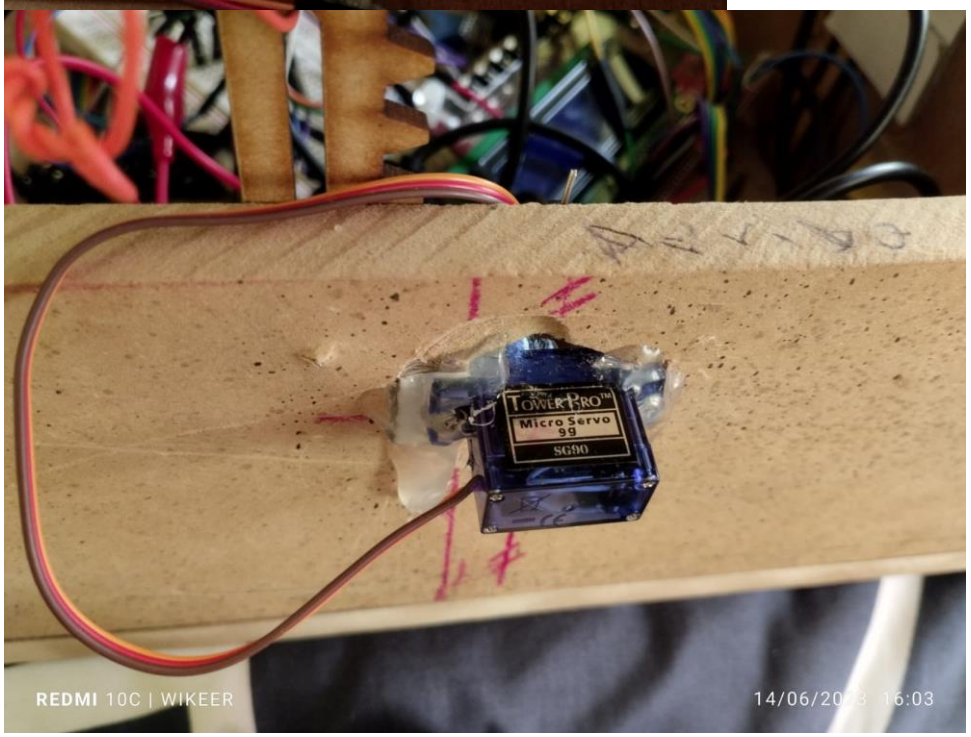

REDMI 10C | WIKEER
14/06/2023 16:04

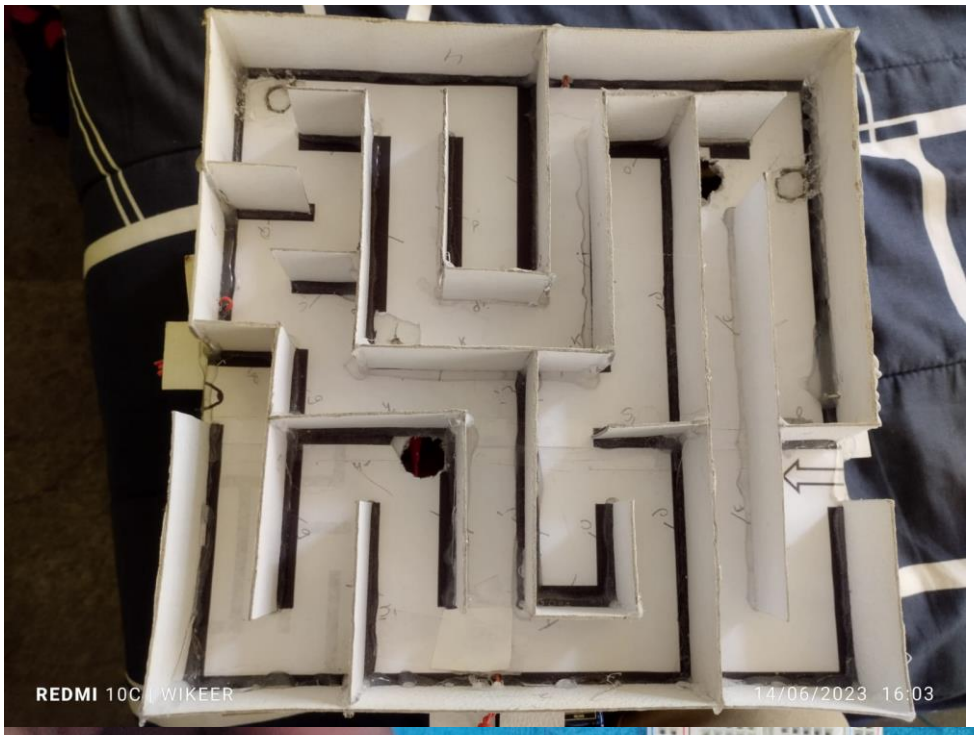
REDMI 10C | WIKEER
14/06/2023 16:04

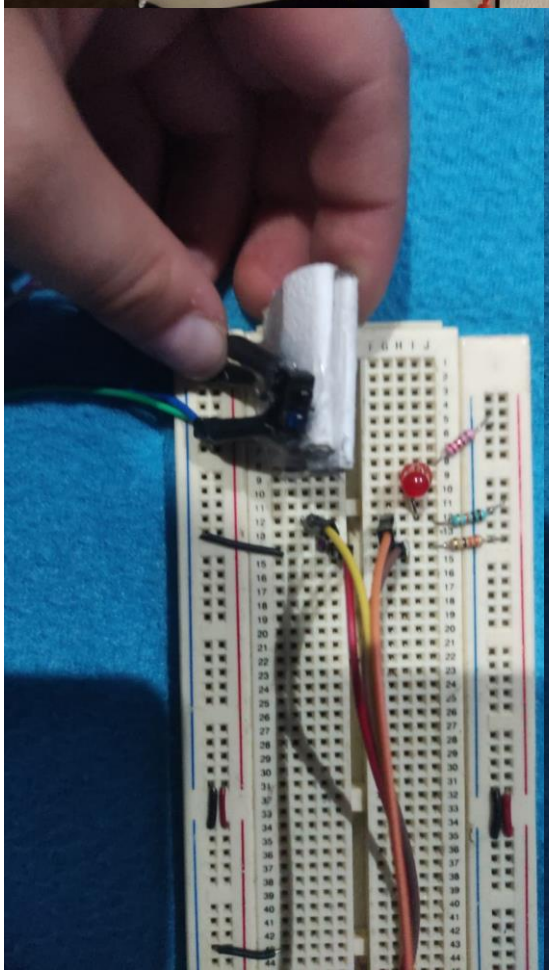

REDMI 10C | WIKEER
14/06/2023 16:04

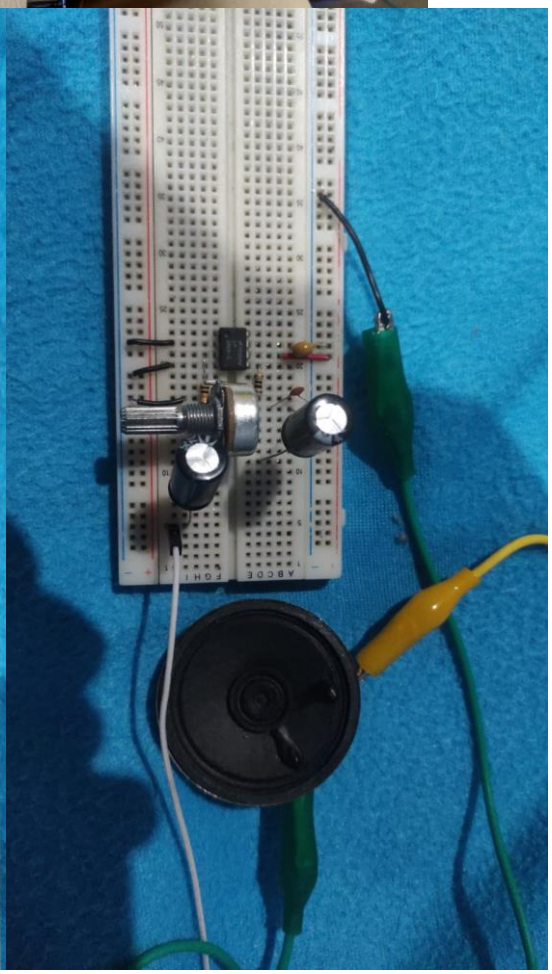
REDMI 10C | WIKEER
14/06/2023 16:04


REDMI 10C | WIKEER
14/06/2023 16:03


TOWER PRO
Micro Servo
99
SG90
REDMI 10C | WIKEER


REDMI 10C | WIKEER
14/06/2023 16:03





CONCLUSIONES

Espinosa Flores Giovanni:
Con este proyecto pudimos demostrar todos los conocimientos adquiridos a durante esta unidad de aprendizaje, desde la implementación y programación de la tarjeta FPGA desde el lenguaje de programación VHDL, hasta la conexión de distintos componentes electrónicos para el control de este proyecto, una de las dificultades con la que nos presentamos fue la correcta comunicación entre el módulo Pmod ACL con los servomotores, ya que estos no realizaban la inclinación del laberinto de forma correcta, este problema se arregló aumentando el tiempo de retardo de comunicación entre el módulo y los servomotores, también aumentando el ancho de pulso que controlaba los servomotores.

Anette Cheyenne Melo Romo
Gracias a este proyecto pudimos implementar diversas cosas vistas a lo largo de todo el curso, así como un sistema mecánico, para esto usamos un sistema de 4 servos acoplados cada uno a una cremallera con su engrane, de tal forma que usando el Pmod ACL comparamos los datos obtenidos y se mandan a través de la tarjeta a nuestros servos, de igual manera gracias al uso de la FPGA podemos hacer varias cosas al mismo tiempo, como es el caso de poner la música, llevar el contador, mandar los PWM a los servomotores y censar la caída de la pelota cuando se pierda
Gracias al modulo Pmod ACL logramos censar el movimiento generado por la mano para así poder controlar el laberinto

Miguel Abraham Rendón Chávez
Gracias al desarrollo de esta practica logramos analizar un codigo externo para despues implementarlo, este siendo del modulo PmodACL que nos lo proporciono el profesor, además de idear el mecanismo para mover el laberinto, tuvimos problemas en esa parte por una mala idealización del codigo para el modulo PmodACL ya que tuvimos problemas con el pwm y la implentacion con la decodificación de grados del modulo, en el que posiblemente cometimos errores al realizar el códigos por esta razón ya no pudimos continuar con lo demas

BIBLIOGRAFIA

PmodACL™ Reference Manual. (2016). Mouser.mx. https://www.mouser.mx/datasheet/2/690/pmodacl_rm-846331.pdf

Sg90 data sheet PDF. (s/f). Datasheetspdf.com. Recuperado el 14 de junio de 2023, de https://datasheetspdf.com/datasheet/search.php?sWord=sg90

(S/f). Mouser.mx. Recuperado el 14 de junio de 2023, de https://www.mouser.mx/ProductDetail/Digilent/410-097?qs=s%2FdyVPQMB4ycs5LfpsPa7Q%3D%3D